

Минобрнауки России
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Санкт-Петербургский государственный технологический институт
(технический университет)»

Направление подготовки	09.03.00 Информатика и вычислительная техника
Факультет	Информационных технологий и управления
Кафедра	Систем автоматизированного проектирования и управления
Курс 1	Группа 4503

КУРСОВОЙ ПРОЕКТ ПО ДИСЦИПЛИНЕ
«ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ И ПРОГРАММИРОВАНИЕ»

Тема: Программный комплекс для изучения основ полиморфизма на примере сортировки матриц (упорядочить каждую строку матрицы по возрастанию четных чисел)

Выполнил обучающийся
Заведующий кафедрой, проф.
Руководитель, доц.
Консультант, ст. преп.

И. Р. Малый
Т. Б. Чистякова
И. Г. Корниенко
А. К. Федин

Минобрнауки России
федеральное государственное бюджетное образовательное учреждение высшего
образования
«Санкт-Петербургский государственный технологический институт
(технический университет)»

ЗАДАНИЕ НА КУРСОВОЙ ПРОЕКТ

Направление подготовки	09.03.01 Информатика и вычислительная техника
Факультет	Информационных технологий и управления
Кафедра	Систем автоматизированного проектирования и управления
Учебная дисциплина	Информационные технологии и программирование
Курс 1	Группа 4503
Студент	Малый Илья Романович

Тема: Программный комплекс для изучения основ полиморфизма на примере сортировки матриц (упорядочить каждую строку матрицы по возрастанию четных чисел)

Цель проектирование и создание прикладного программного обеспечения,
работы: позволяющего решать задачу сортировки динамических матриц, а также проводить сравнение методов сортировки.

Исходные данные по проекту:

- 1 Чистякова, Т. Б. Программирование на языках высокого уровня. Базовый курс / Т. Б. Чистякова, Р. В. Антипин, И. В. Новожилова. – Санкт-Петербург : СПбГТИ(ТУ), 2008. – 227 с. – ISBN 978-5-91884-017-7.
- 2 Павловская, Т. А. C/C++. Программирование на языке высокого уровня / Т. А. Павловская. – Санкт-Петербург: Питер, 2021. – 461 с. – ISBN 978-5-4461-3916-3.
- 3 Лафоре, Р. Объектно-ориентированное программирование в C++ / Р. Лафоре. – 4-е изд. – Санкт-Петербург : Питер, 2022. – 928 с. – ISBN 978-5-4461-0927-2.
- 4 Иванова, Г. С. Объектно-ориентированное программирование / Г. С. Иванова, Т. Н. Ничушкина, Е. К. Пугачев. – Москва : МГТУ, 2007. – 366 с. – ISBN 978-5-7038-2775-8.
- 5 Гутгарц, Р. Д. Проектирование автоматизированных систем обработки информации и управления : учебник для вузов / Р. Д. Гутгарц. – 2-е изд., перераб. и доп. – Москва : Издательство Юрайт, 2026. – 351 с. – (Высшее образование). – ISBN 978-5-534-15761-1.
- 6 Алгоритмы. Построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн. – 3-е изд. – Москва : Вильямс, 2013. – 1328 с. – ISBN 978-5-8459-1794-6.
- 7 Седжвик, Р. Фундаментальные алгоритмы на C++. Анализ. Структуры данных. Сортировка. Поиск. / Р. Седжвик – Киев : ДиаСофт, 2001. – 688 с. – ISBN 966-7393-89-5.

Перечень вопросов, подлежащих разработке:

- 1 Аналитический обзор.
 - 1.1 Матричные вычисления и компьютерные системы для работы с ними. Сравнительная характеристика существующих систем-аналогов (Mathcad 15.0, Excel 2016, Scilab 2026.1.0). Обоснование актуальности разработки программного комплекса для изучения основ полиморфизма на примере сортировки матриц.
 - 1.2 Общая характеристика и особенности процесса сортировки матриц.

- 1.3 Обзор и обоснование выбора инструментальных средств разработки программного комплекса для изучения основ полиморфизма на примере сортировки матриц.
- 2 Цель и задачи курсового проекта.
- 3 Технологическая часть.
 - 3.1 Формализованное описание процесса сортировки матриц как объекта обработки информации.
 - 3.2 Постановка задачи обработки информации.
 - 3.3 Разработка функциональной структуры программного комплекса для изучения основ полиморфизма на примере сортировки матриц.
 - 3.4 Создание алгоритма сортировки матриц (упорядочить каждую строку матрицы по возрастанию четных чисел).
 - 3.5 Разработка структуры интерфейсов для пользователя программного комплекса.
 - 3.6 Описание структур данных и алгоритмов (формат представления данных).
 - 3.7 Описание структуры программы (модули, основные функции).
 - 3.8 Тестирование программного комплекса (на заданном примере).
 - 3.9 Характеристика программного и аппаратного обеспечения.
- Оформление документации (пояснительной записки, презентации) по проекту.

Перечень графического материала:

- 1 Формализованное описание процесса сортировки матриц.
- 2 Функциональная структура программного комплекса для изучения основ полиморфизма на примере сортировки матриц.
- 3 Блок-схемы алгоритма сортировки матриц (упорядочить каждую строку матрицы по возрастанию четных чисел).
- 4 UML-диаграмма вариантов использования для пользователя системы.
- 5 Тестовый пример работы программного комплекса.
- 6 Характеристика программного и аппаратного обеспечения.

Требования к аппаратному и программному обеспечению:

Аппаратное обеспечение: шестиядерный процессор Ryzen 5 5600 с тактовой частотой 4 ГГц, 16 ГБ оперативной памяти DDR4, твердотельный накопитель SSD NVMe 2 ТБ, графический процессор AMD Radeon RX 6600.

Программное обеспечение: операционная система Arch Linux, рабочая среда KDE 5, среда разработки CLion 2025.3.3, офисный пакет Onlyoffice 9.0.4.50, включающий в себя средство для оформления документов и презентаций.

Дата выдачи задания:

Дата предоставления курсового проекта к защите:

Руководитель, доц.

И. Г. Корниенко

Консультант, ст. преп.

А. К. Федин

Задание принял к выполнению

И. Р. Малый

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	6
1 Аналитический обзор.....	7
1.1 Матричные вычисления и компьютерные системы для работы с ними. Сравнительная характеристика существующих систем-аналогов. Обоснование актуальности разработки программного комплекса для изучения основ полиморфизма на примере сортировки матриц.....	7
1.2 Общая характеристика и особенности процесса сортировки матриц.....	12
1.3 Обзор и обоснование выбора инструментальных средств разработки программного комплекса для изучения основ полиморфизма на примере сортировки матриц.....	13
2 Цель и задачи курсового проекта.....	14
3 Технологическая часть.....	15
3.1 Формализованное описание процесса сортировки матриц как объекта обработки информации.....	15
3.2 Постановка задачи обработки информации.....	16
3.3 Разработка функциональной структуры программного комплекса для изучения основ полиморфизма на примере сортировки матриц.....	16
3.4 Создание алгоритма сортировки матриц.....	17
3.5 Разработка структуры интерфейсов для пользователя программного комплекса.....	23
3.6 Описание структур данных и алгоритмов.....	25
3.7 Описание структуры программы.....	26
3.8 Тестирование программного комплекса.....	29
3.9 Характеристика программного и аппаратного обеспечения.....	33
ВЫВОДЫ ПО ПРОЕКТУ	36
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37

ВВЕДЕНИЕ

В мире программирования существует множество языков программирования. С каждым годом их становится всё больше и больше. Наиболее известными и распространёнными языками являются C++, C#, Java, Python, Pascal, PHP. Этими языками пользуются, так как они удобны для программирования, и они могут работать с комплексными структурами данных.

Наибольшей популярностью пользуются языки с объектно-ориентированными возможностями. Объектно-Ориентированное Программирование (далее ООП) – это парадигма разработки программных систем, в которой проекты состоят из объектов и классов. Объектно-Ориентированное Программирование стало неотъемлемой частью разработки многих современных проектов. Оно способствует лучшей управляемости проектом во время разработки и его тестированию. Код таких проектов легко читается и быстро пишется.

Одним из трех главных принципов ООП является полиморфизм. Полиморфизм – слово греческого происхождения, означающее "многообразие форм". В программировании означает – способность программы идентично использовать объекты с одинаковым интерфейсом без информации о конкретном типе этого объекта

Матричные вычисления – это область прикладной математики, которая занимается операциями над матрицами (массивами чисел в виде строк и столбцов) для решения практических задач в инженерии, науке и анализе данных. К основным операциям относятся: сложение, вычитание, умножение матриц, транспонирование, вычисление определителя, поиск собственных значений и решение систем линейных уравнений. Эти операции являются основой для моделирования физических процессов, обработки больших объемов данных, криптографии и компьютерной графики.

Алгоритмы сортировки – это фундаментальные методы упорядочивания данных, которые различаются по принципу работы, временной сложности и практической эффективности. Ввиду этого различия следует, что наиболее оптимальный алгоритм разнится для каждой матрицы, а продукт данной работы позволит его определить.

В данной работе будет создан программный комплекс, который позволит отсортировать матрицу, используя разные алгоритмы сортировки, и получить подробную статистику о количестве проведенных операций в процессе сортировки. При разработке будут использованы принципы ООП (по большей степени полиморфизм), что позволит подробно изучить их, а также увидеть практическую необходимость их существования.

1 Аналитический обзор

1.1 Матричные вычисления и компьютерные системы для работы с ними. Сравнительная характеристика существующих систем-аналогов. Обоснование актуальности разработки программного комплекса для изучения основ полиморфизма на примере сортировки матриц

На современном рынке существуют мощные специализированные инструменты, которые оптимизированы для работы с матрицами. Некоторые из них будут рассмотрены ниже

Microsoft Excel – электронная таблица, которая хотя и не предназначена специально для матричных вычислений, широко используется благодаря доступности и простоте. В Excel можно выполнять матричные операции через формулы, что делает его популярным в бизнесе и экономике. Excel имеет платную лицензию, и доступен только на операционной системе (далее – ОС) Windows. При работе с большими матрицами (больше 10000 ячеек) производительность снижается. В основном используется в бухгалтерских расчетах, поэтому для больших матричных вычислений не оптимизирован. На рисунках 1, 2 представлен интерфейс данной программы, а также некоторые операции с матрицами (транспонирование, сложение, умножение).

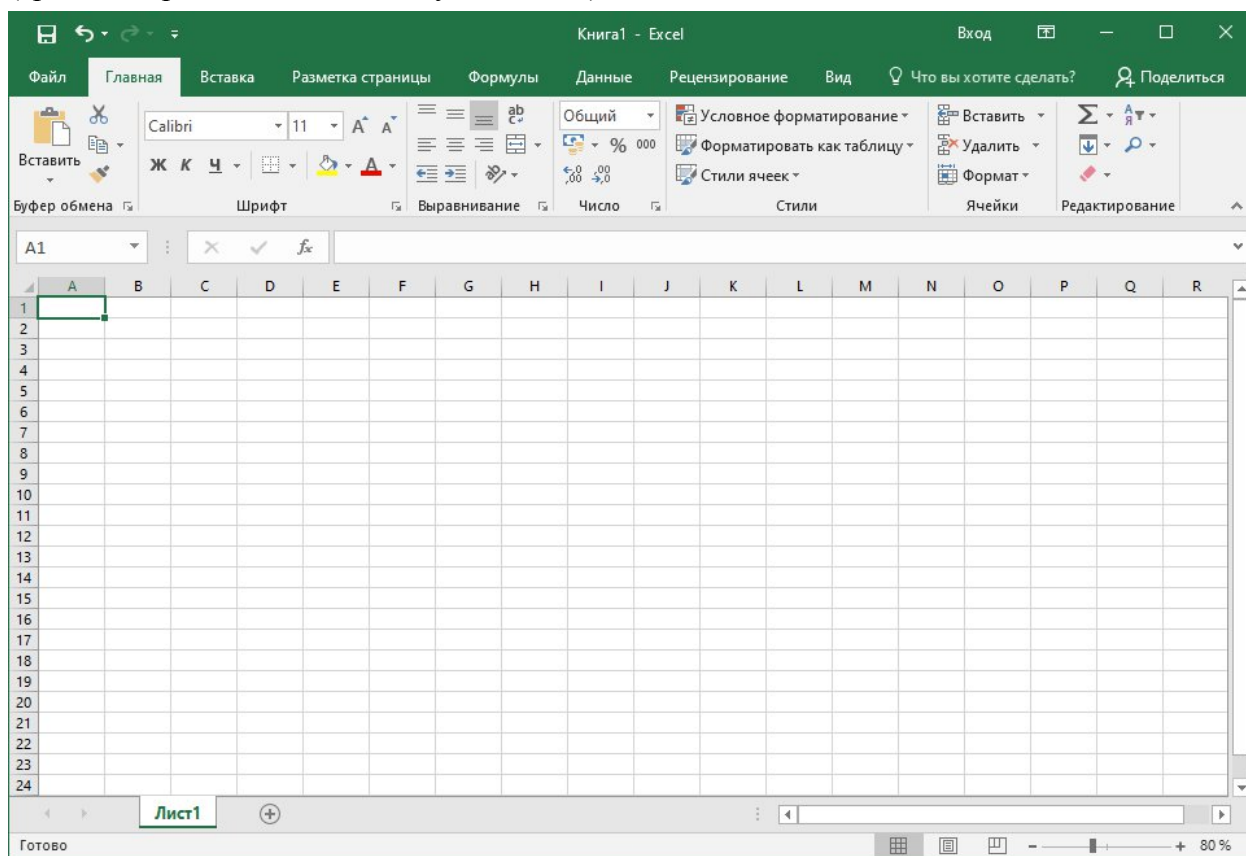


Рисунок 1 – Интерфейс Microsoft Excel

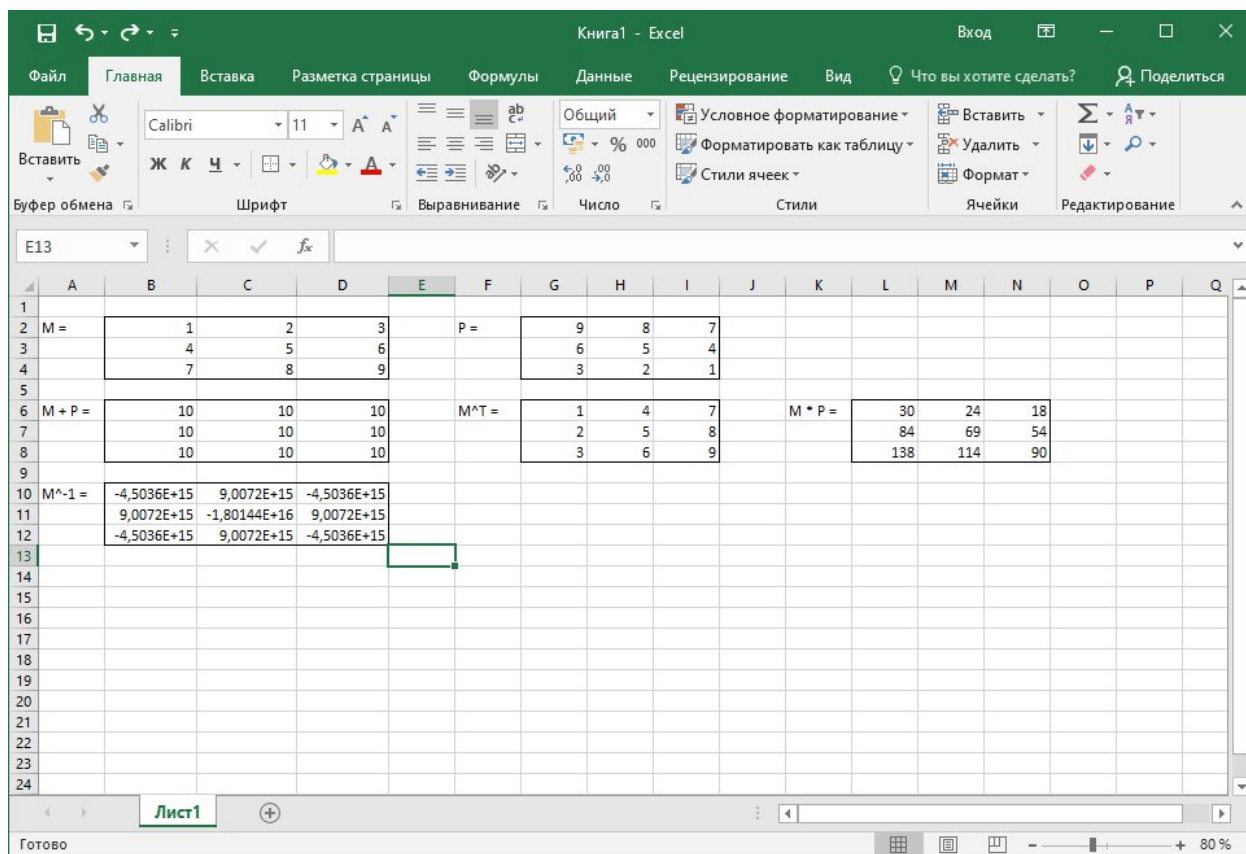


Рисунок 2 – Математические операции с матрицами в Excel

Следующее решение, которое будет рассмотрено в данной работе – PTC Mathcad. Mathcad – интерактивная система компьютерной математики, позволяющая проводить аналитические и численные расчеты в удобном графическом интерфейсе. Mathcad предоставляет встроенные функции для всех стандартных матричных операций и хорошо подходит для инженерных расчетов. Также, как и Excel, имеет платную лицензию и доступен только на ОС Windows. Хорошо оптимизирован для сложных инженерных расчетов. На рисунках 3, 4 представлен интерфейс программы и некоторые операции с матрицами.

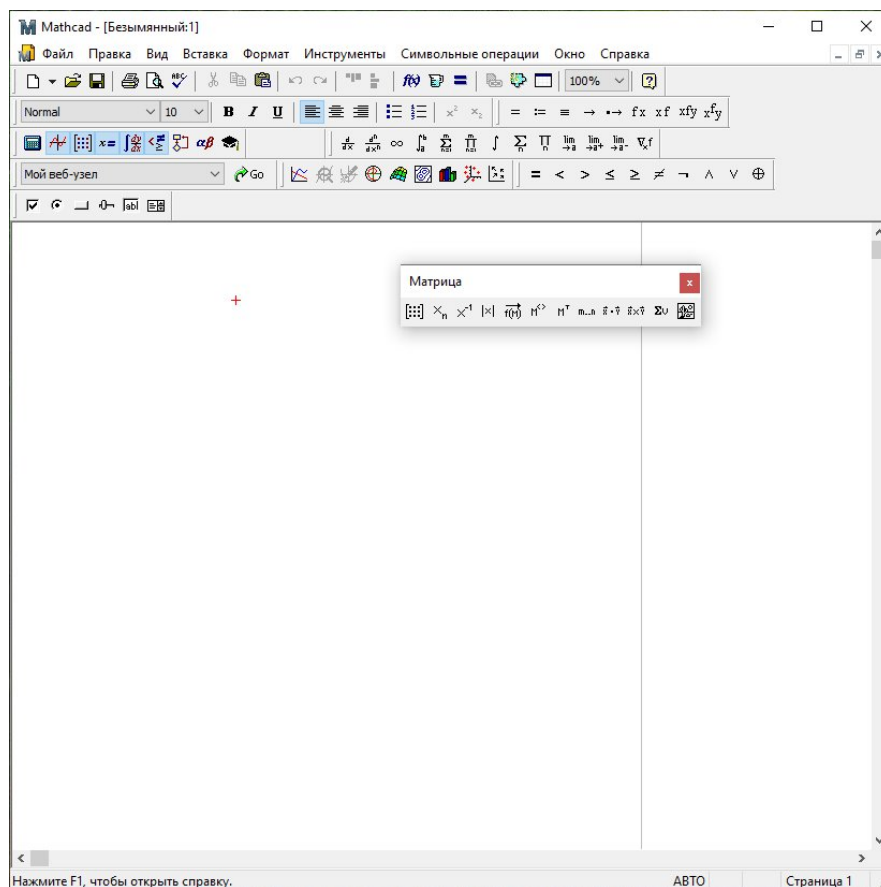


Рисунок 3 – Интерфейс Mathcad

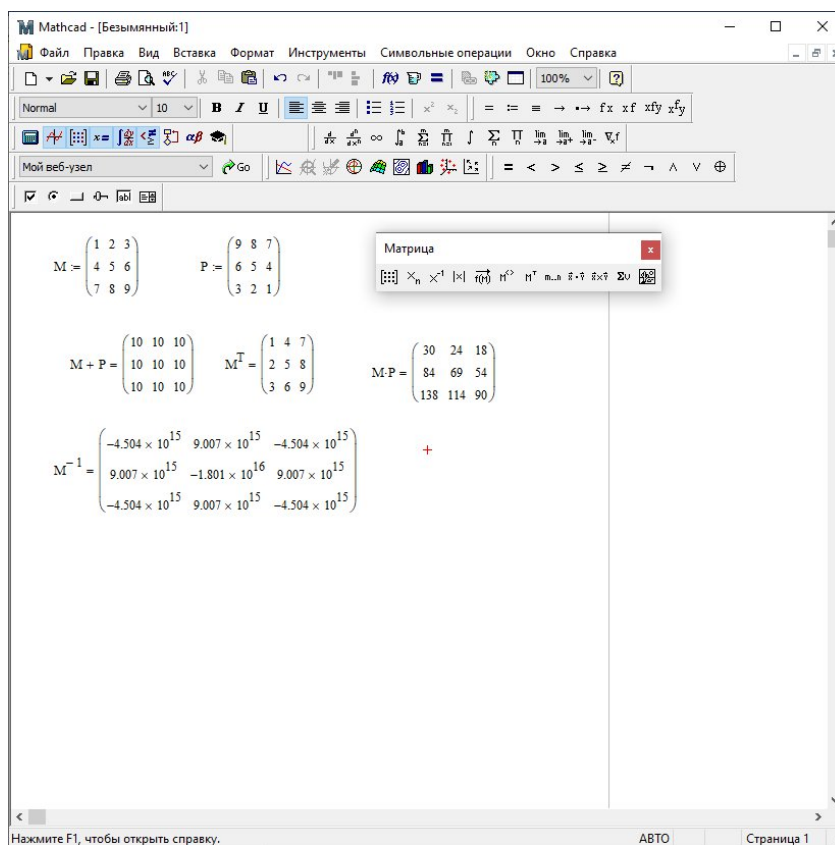


Рисунок 4 – Математические операции с матрицами в Mathcad

Последним решением, рассмотренным в данном проекте, будет Scilab. Scilab – свободная система для численных вычислений, аналогичная MATLAB. Она имеет собственный язык программирования, оптимизирована для матричных операций и часто используется в научных исследованиях и образовательных учреждениях. В отличие от предыдущих программ имеет свободную лицензию и открытый исходный код и поддерживает ОС Windows, Linux и MacOS. Хорошо оптимизирован для больших матричных расчетов. Интерфейс программы и некоторые матричные операции представлены на рисунках 5-7.

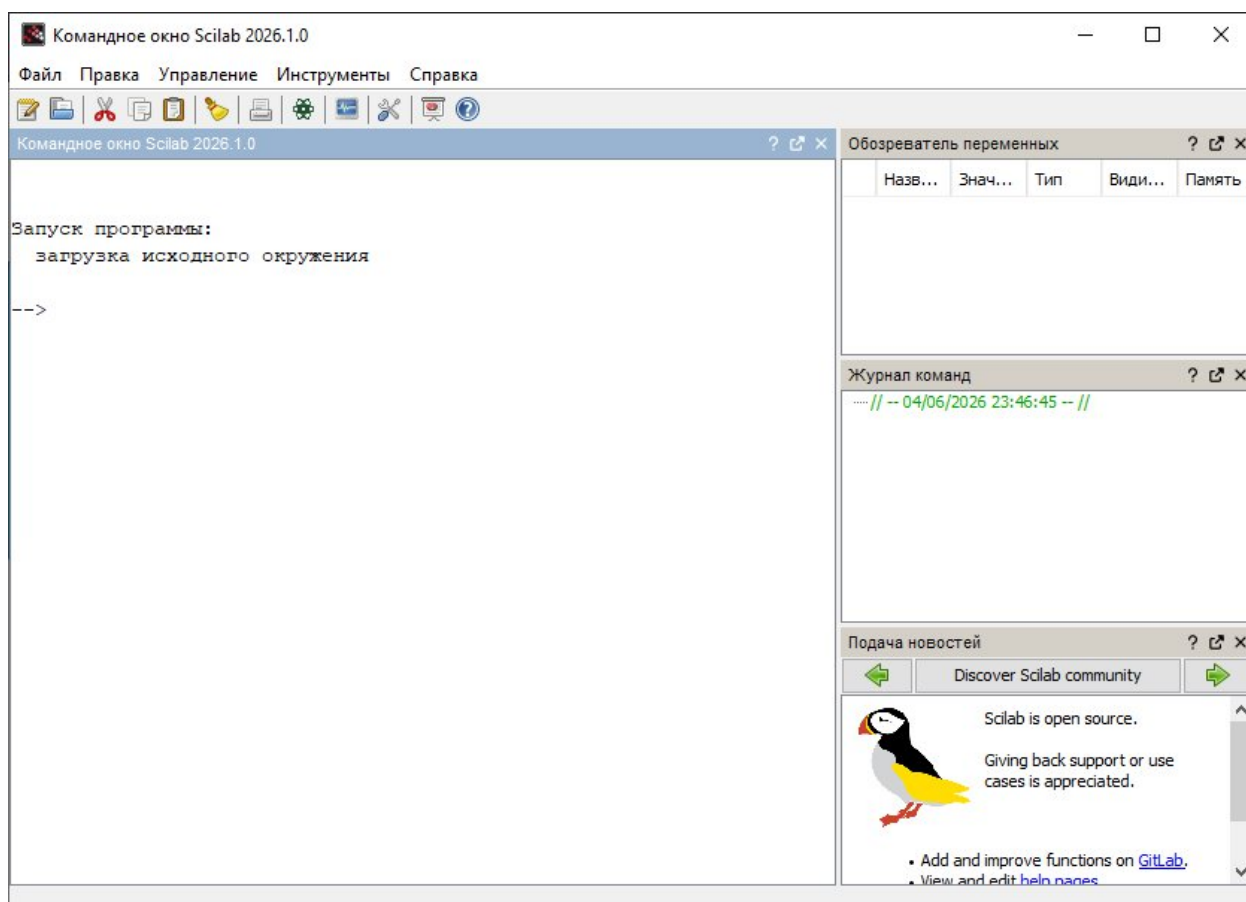


Рисунок 5 – Интерфейс Scilab

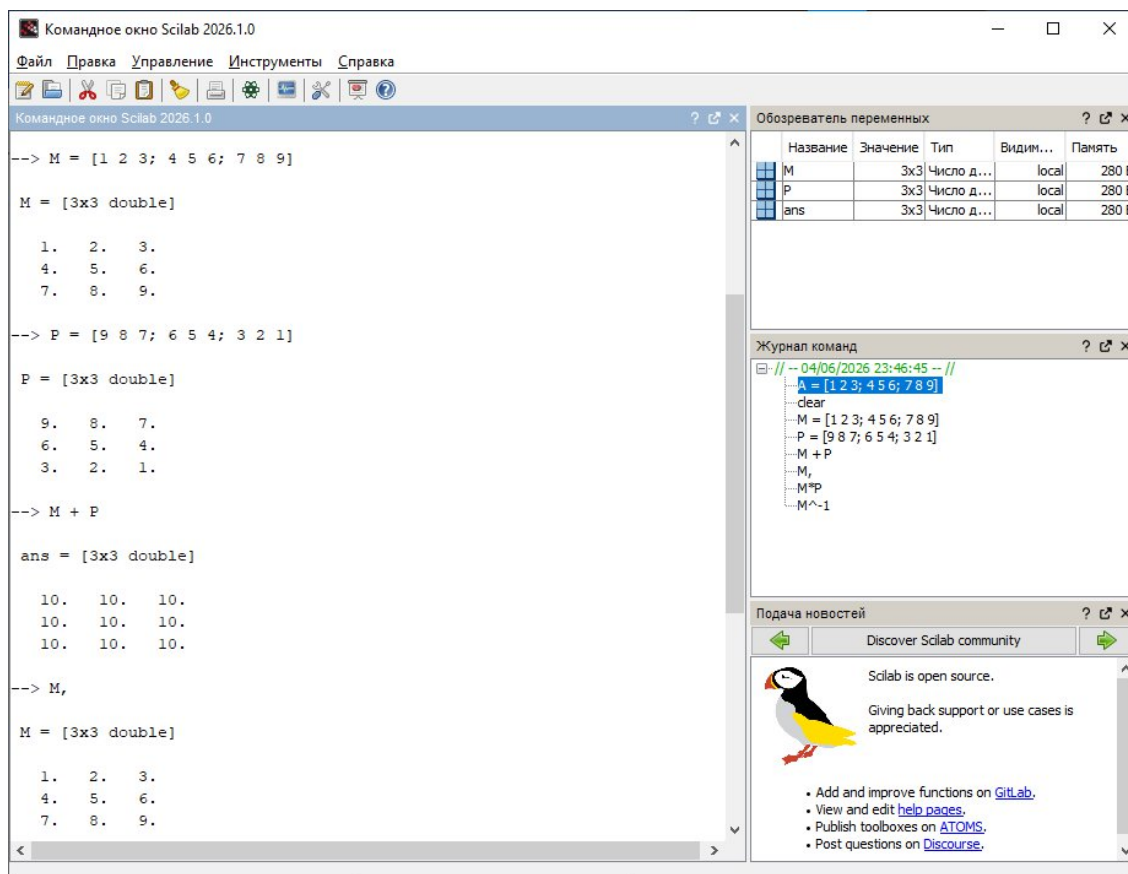


Рисунок 6 – Математические операции с матрицами в Scilab

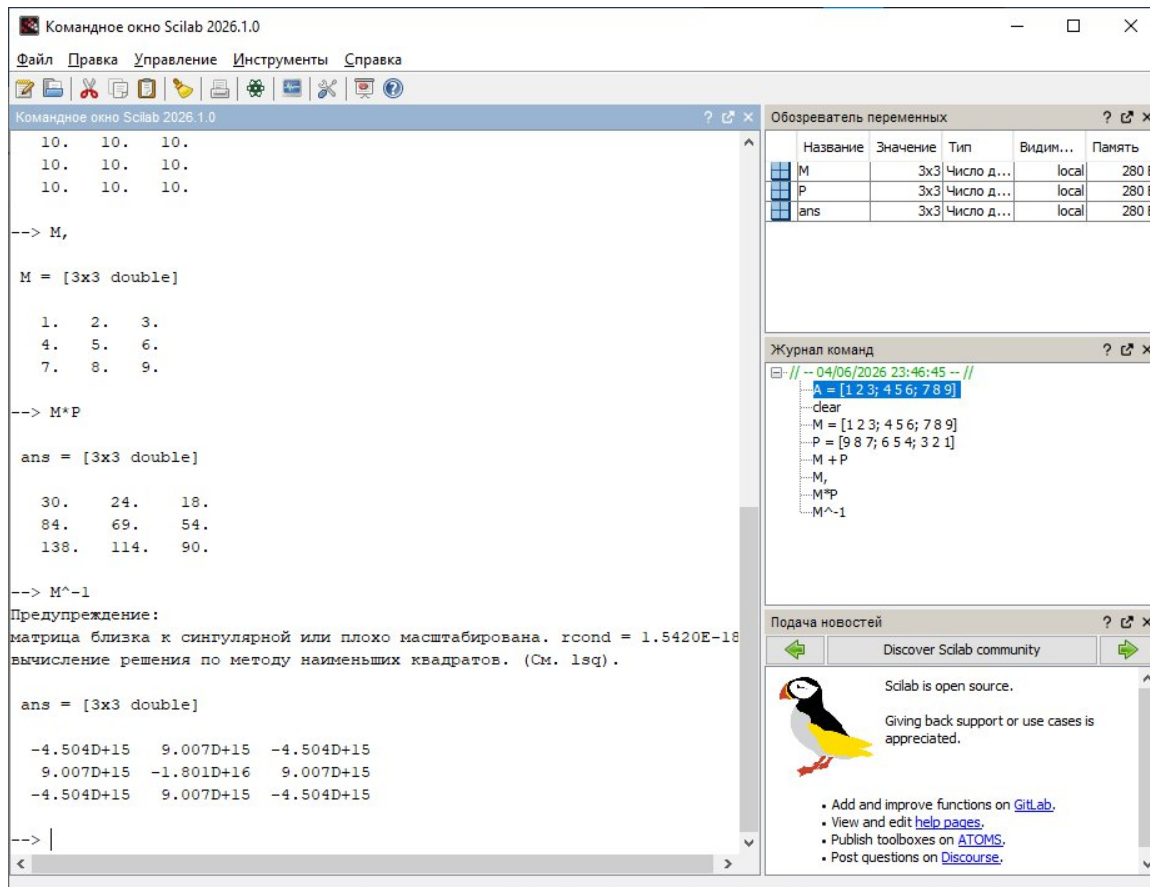


Рисунок 7 – Математические операции с матрицами в Scilab

Выбранные три решения были сравнены по следующим характеристикам: основное назначение, тип лицензии, поддержка ОС, сложность внедрения, встроенный язык программирования, поддерживаемые матричные операции. Сравнение было приведено в таблице 1.

Таблица 1 – Сравнительная характеристика выбранных решений

Критерий сравнения	Microsoft Excel	PTC Mathcad	Scilab
Основное назначение	Бухгалтерские расчеты	Инженерные расчёты	Инженерные расчёты
Тип лицензии	Коммерческая (платная)	Коммерческая (платная)	Свободная (бесплатная)
Поддержка ОС	Windows	Windows	Windows, Linux, MacOS
Сложность внедрения	Низкая, вычисления производятся с помощью интуитивно понятных формул, имеется подробная справка	Низкая, вычисления записываются как в математике, интерфейс интуитивно понятен, имеется подробная справка	Средняя, интерфейс программы – консольный, что может затруднить использование, имеется подробная документация
Встроенный язык программирования	Встроенные формулы, Visual Basic	Встроенные логические операторы и циклы	Встроенный язык сценариев
Поддерживаемые матричные операции	Базовые арифметические ¹	Базовые арифметические, сортировка столбцов/строк по возрастанию/убыванию	Базовые арифметические, вычисление следа и ранга матрицы, приведение к треугольной форме.

Данные программные решения предоставляют возможность производить матричные вычисления, в Mathcad даже доступны базовые сортировки, однако это не является их главной проблематикой, они не предоставляют возможности выполнить нашу задачу – отсортировать матрицу по определенному принципу и определить наиболее эффективный алгоритм сортировки. Этим и обусловлена актуальность данной работы.

1.2 Общая характеристика и особенности процесса сортировки матриц

Сортировка – процесс перестановки элементов таким образом, чтобы для каждого элемента в некой последовательности выполнялось единое условие неравенства по отношению к следующему элементу последовательности.

Обычно объектом сортировки является одномерная последовательность и данное выше определение очевидно для одномерных структур. Однако матрицы по своей структуре двумерные, поэтому процесс сортировки матрицы не однозначен. Варианты как можно отсортировать матрицу:

- а) сортировать каждую строку независимо от других строк,

¹ Под базовыми арифметическими имеются ввиду операции: матричного сложения, вычитания, произведения, нахождения определителя, обратной матрицы, транспонирования

- б) сортировать каждый столбец независимо от других столбцов,
- в) преобразовать матрицу в одномерную последовательность, отсортировать её, затем преобразовать её обратно в матрицу,
- г) расставить строки в отсортированном порядке, относительно поэлементного сравнения строк, не изменяя сами строки,
- д) расставить столбцы в отсортированном порядке, относительно поэлементного сравнения столбцов, не изменяя сами столбцы.

В рамках данной работы будет рассмотрен тип сортировки а, такой вариант используется, когда каждая строка имеет самостоятельный смысл и строки не зависят друг от друга. Практический пример: распределения товаров в строке заказа по цене или иному показателю,

1.3 Обзор и обоснование выбора инструментальных средств разработки программного комплекса для изучения основ полиморфизма на примере сортировки матриц

Для реализации поставленной задачи разработано программное обеспечение, включающее графический пользовательский интерфейс. Для написания кода программы был выбран язык C++. В качестве интегрированной среды разработки программного обеспечения (Integrated Development Environment, IDE) используется CLion 2025.3.3 .

Язык программирования C++ обеспечивает близкий к машинному коду уровень управления памятью и ресурсами, что критично, при обработке больших объемах данных. Кроме того, C++ разрабатывался как объектно-ориентированный язык, это делает изучение полиморфизма возможным. Также к преимуществам можно отнести широкую поддержку платформ, что позволяет компилировать и запускать разрабатываемые программы на различных устройствах и ОС.

Среда CLion была выбрана по следующим причинам:

- Поддержка последних стандартов языка разработки C++,
- Встроенная поддержка CMake, что позволяет облегчить сборку проекта,
- Подсказки типов – позволяет ускорить написание кода,
- Бесплатная лицензия для некоммерческого использования.

На рисунках 8, 9 представлен интерфейс данной программы.

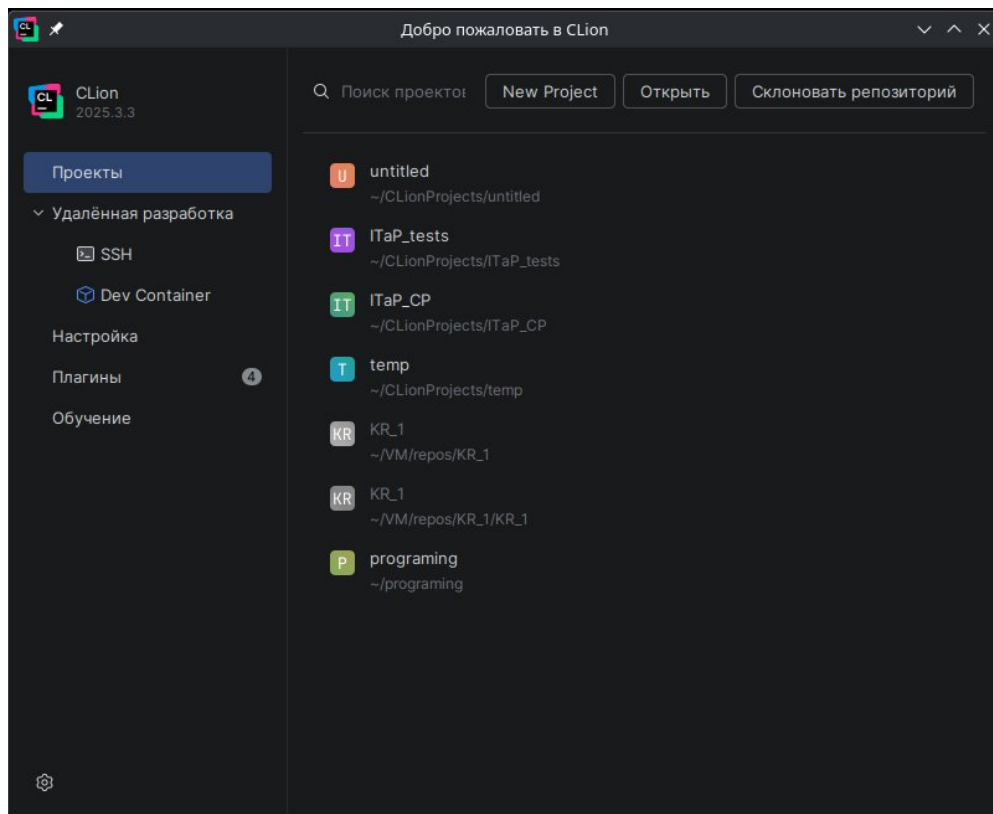


Рисунок 8 – Главная страница CLion

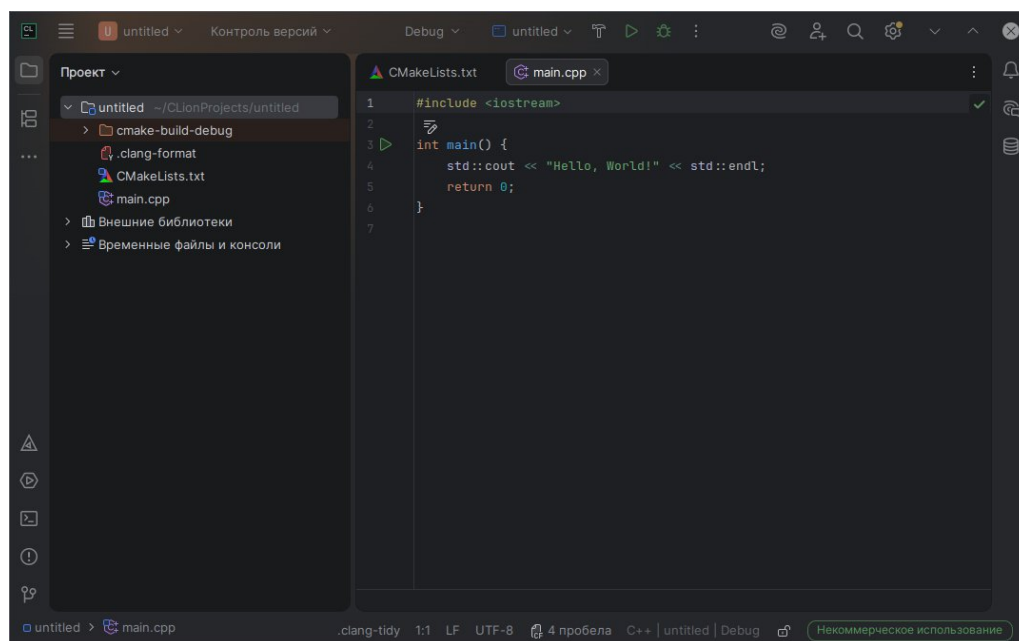


Рисунок 9 – Пустой проект CLion

2 Цель и задачи курсового проекта

Целью данного проекта – проектирование и создание прикладного программного комплекса, использующего полиморфизм в своей основе, позволяющего решать задачу сортировки динамических матриц по заданному принципу, а также проводить сравнение методов сортировки.

Для достижения цели были выполнены следующие задачи:

- а) изучить существующие алгоритмы сортировок,
- б) выполнить постановку задачи,
- в) составить формализованное описание процесса сортировки, разработать структуру входных и выходных данных,
- г) разработать алгоритм (блок-схему) для сортировки матрицы и для каждого алгоритма сортировки,
- д) спроектировать структуру программного комплекса,
- е) разработать пользовательский интерфейс программного комплекса,
- ж) разработать программу, реализующую поставленную задачу,
- з) протестировать работоспособность программного обеспечения.

3 Технологическая часть

3.1 Формализованное описание процесса сортировки матриц как объекта обработки информации

На рисунке 10 изображена схема – формализованное описание процесса сортировки. На вход подается произвольная матрица, а на выходе получается отсортированная матрица и количество сравнений и перестановок, совершенных в процессе сортировки.

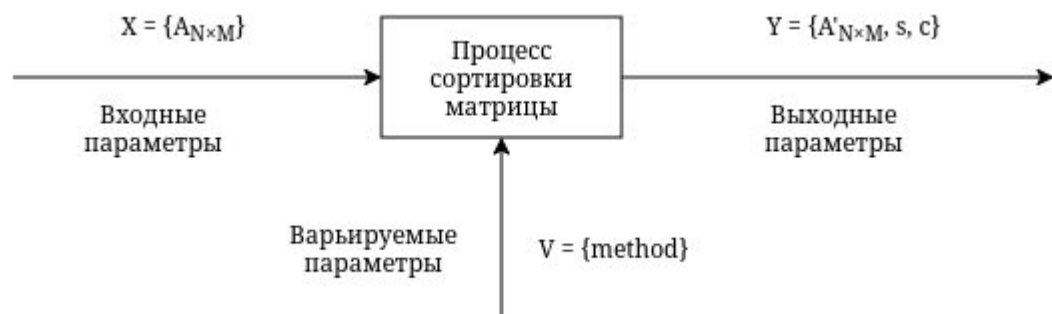


Рисунок 10 – Формализованное описание программного комплекса

Где:

$A_{N \times M}$ – исходная матрица,

N – количество строк матрицы,

M – количество столбцов матрицы,

method – алгоритм сортировки, один из:

- пузырьковая сортировка,
- сортировка выбором,
- сортировка вставкой,
- сортировка Шелла,
- быстрая сортировка

$A'_{N \times M}$ – отсортированная матрица,

s – количество перестановок,
 c – количество сравнений.

3.2 Постановка задачи обработки информации

Для введенной матрицы путем варьирования алгоритмов, получить отсортированную матрицу, выявить наиболее оптимальный метод. Под наиболее оптимальным будем понимать тот алгоритм, используя который сумма сравнений и перестановок будет минимальна.

3.3 Разработка функциональной структуры программного комплекса для изучения основ полиморфизма на примере сортировки матриц

Самыми важными структурными компонентами программы являются модули, в которых находится программный код, и связанные с ними экранные формы. Программный код модуля состоит из взаимосвязанных подпрограмм (процедур), каждая из которых выполняет какую-либо конкретную задачу.

На рисунке 11 изображена функциональная структура разрабатываемого программного комплекса.

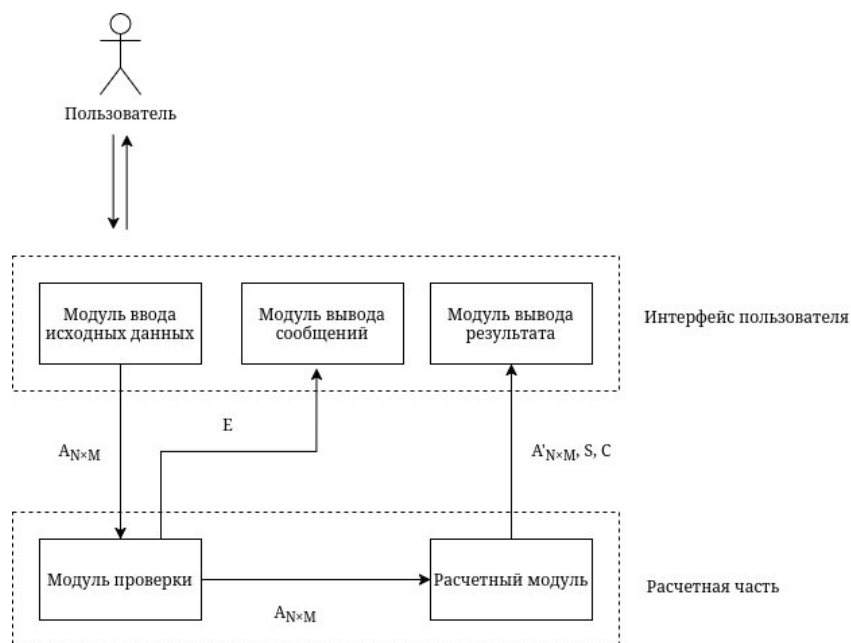


Рисунок 11 – Функциональная структура программного комплекса

Где:

$A_{N \times M}$ – исходная матрица,

N – количество строк матрицы,

M – количество столбцов матрицы,

E – вектор сообщений об ошибках,

$A'_{N \times M}$ – отсортированная матрица,

S – вектор количества перестановок для каждого алгоритма,

C – вектор количества сравнений для каждого алгоритма.

Описание модулей функциональной структуры:

- модуль ввода исходных данных получает от пользователя матрицу $A_{N \times M}$;

- модуль проверки проверяет введенные данные на корректность, например, если размер не является натуральным числом, если возникают ошибки – передает их в модуль вывода сообщений, в ином случае – передает матрицу в расчетный модуль;

- модуль вывода ошибок выводит сообщения об ошибках в векторе E , чтобы пользователь мог их исправить;

- расчетный модуль сортирует полученную матрицу используя разные алгоритмы, собирает количество операций, и передает их в модуль вывода результата в виде двух векторов S и C , также передает отсортированную матрицу;

- модуль вывода результата анализирует суммарное количество операций по каждому методу, выводит какой из них оказался наиболее оптимальным, статистику по количеству операций остальных методов и отсортированную матрицу.

3.4 Создание алгоритма сортировки матриц

Общий алгоритм сортировки матрицы:

- пройти по каждой строке матрицы:

- а) пройти по каждому элементу строки:

- 1) если элемент четный: скопировать его во временный вектор T ;

- б) отсортировать вектор T ;

- в) Пройти по каждому элементу строки:

- 1) если элемент четный: заменить его соответственным элементом из вектора T ;

- г) очистить вектор T ;

Блок-схема данного алгоритма изображена на рисунке 12.

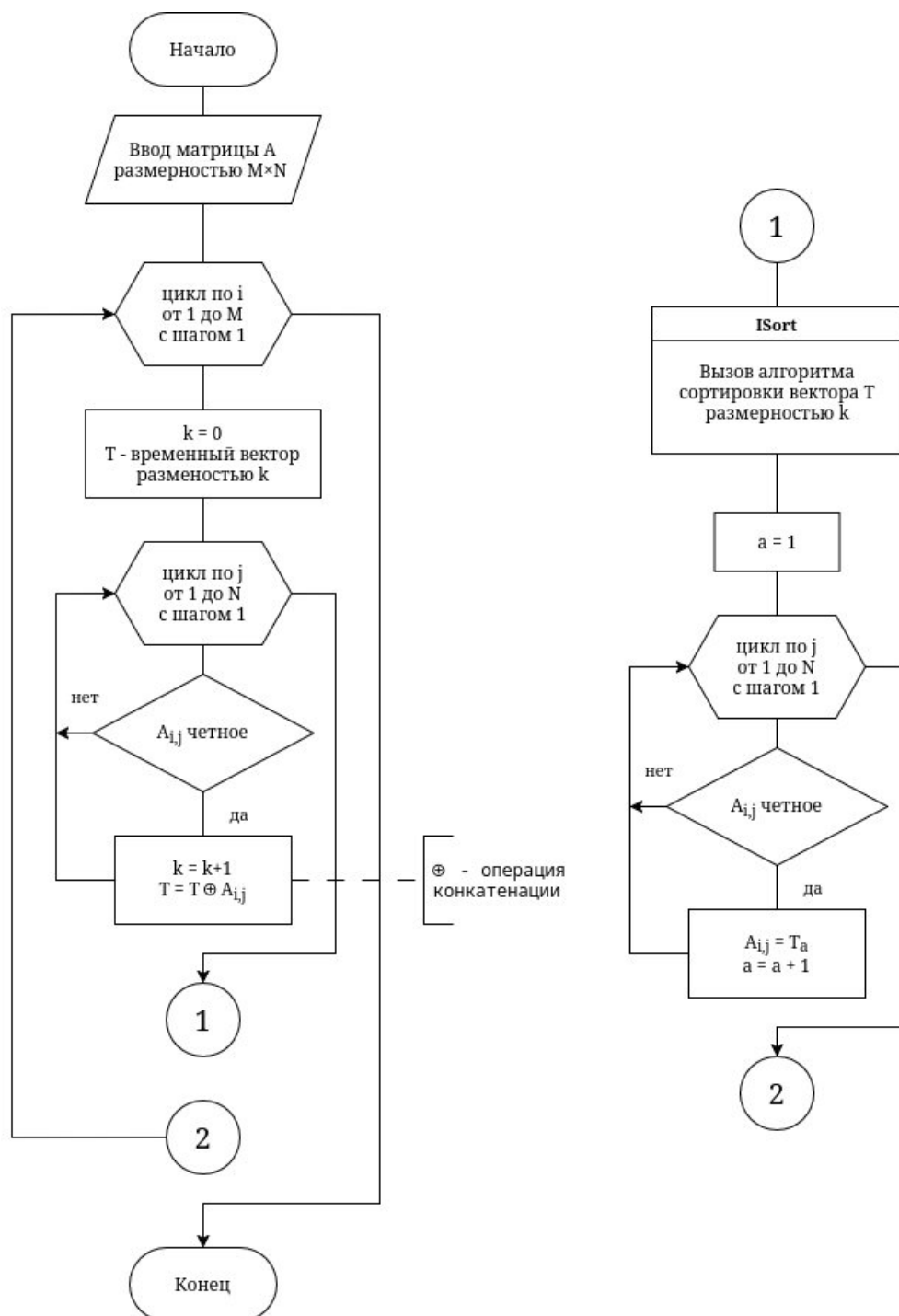


Рисунок 12 – Блок-схема алгоритма сортировки матрицы

Далее рассмотрим алгоритмы сортировок.

Основной принцип пузырьковой сортировки заключается в сравнении соседних элементов и перестановки их если они в неправильном порядке, повторять это до тех пор, пока все элементы не будут отсортированы.

Блок-схема пузырьковой сортировки изображена на рисунке 13.

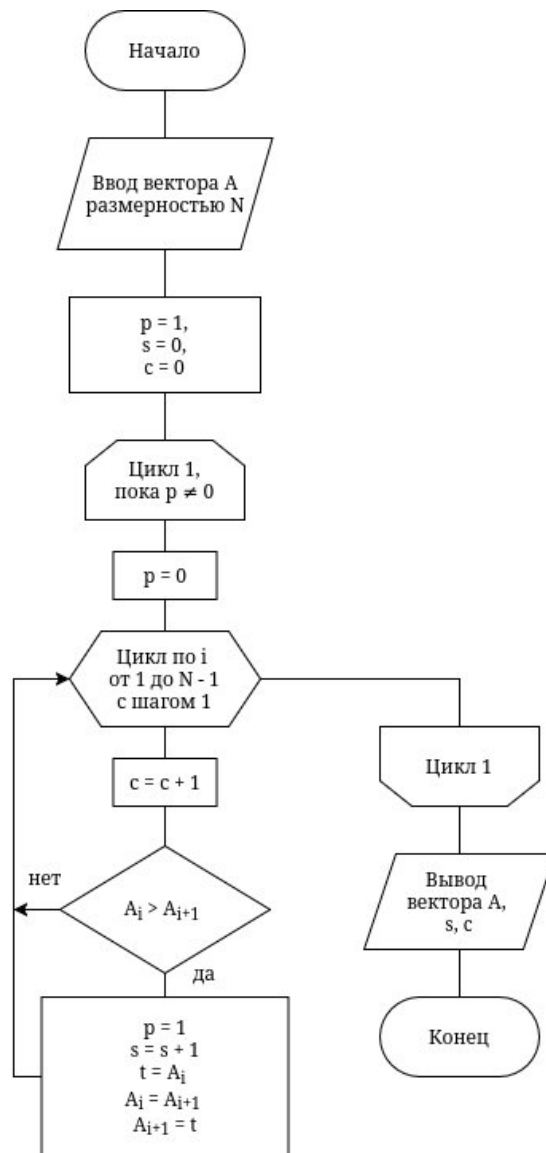


Рисунок 13 – Блок-схема алгоритма пузырьковой сортировки

Сортировка выбором работает по принципу выбора наименьшего элемента из не отсортированной части массива и установки его в конец отсортированной части.

Блок-схема алгоритма сортировки выбором изображена на рисунке 14.

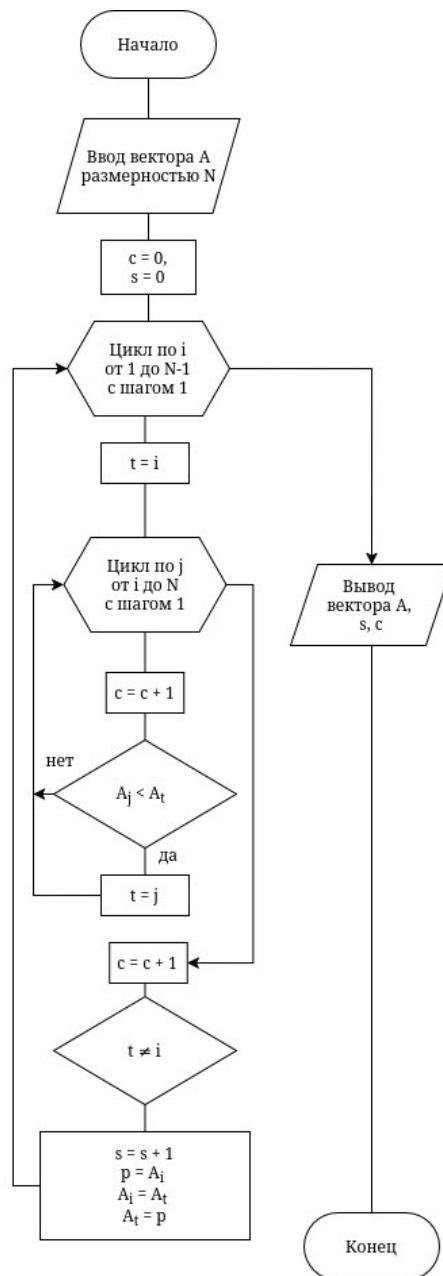


Рисунок 14 – Блок-схема алгоритма сортировки выбором

В алгоритме сортировки вставкой элементы просматриваются по одному и устанавливаются на подходящее место в отсортированной части массива.

Блок-схема алгоритма сортировки вставкой изображена на рисунке 15.

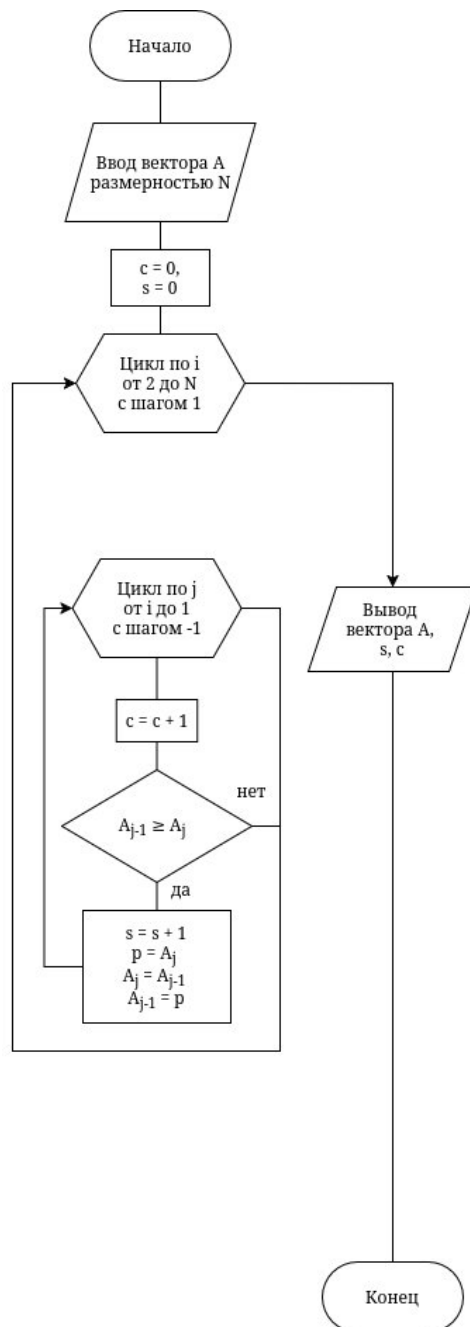


Рисунок 15 – Блок-схема алгоритма сортировки вставкой

Алгоритм сортировки Шелла работает по принципу выбора каждые h -тых элементов массива и сортировки их вставкой, где h берется равным половине длине массива и уменьшается в два раза с каждым проходом.

Блок-схема алгоритма сортировки Шелла изображена на рисунке 16.

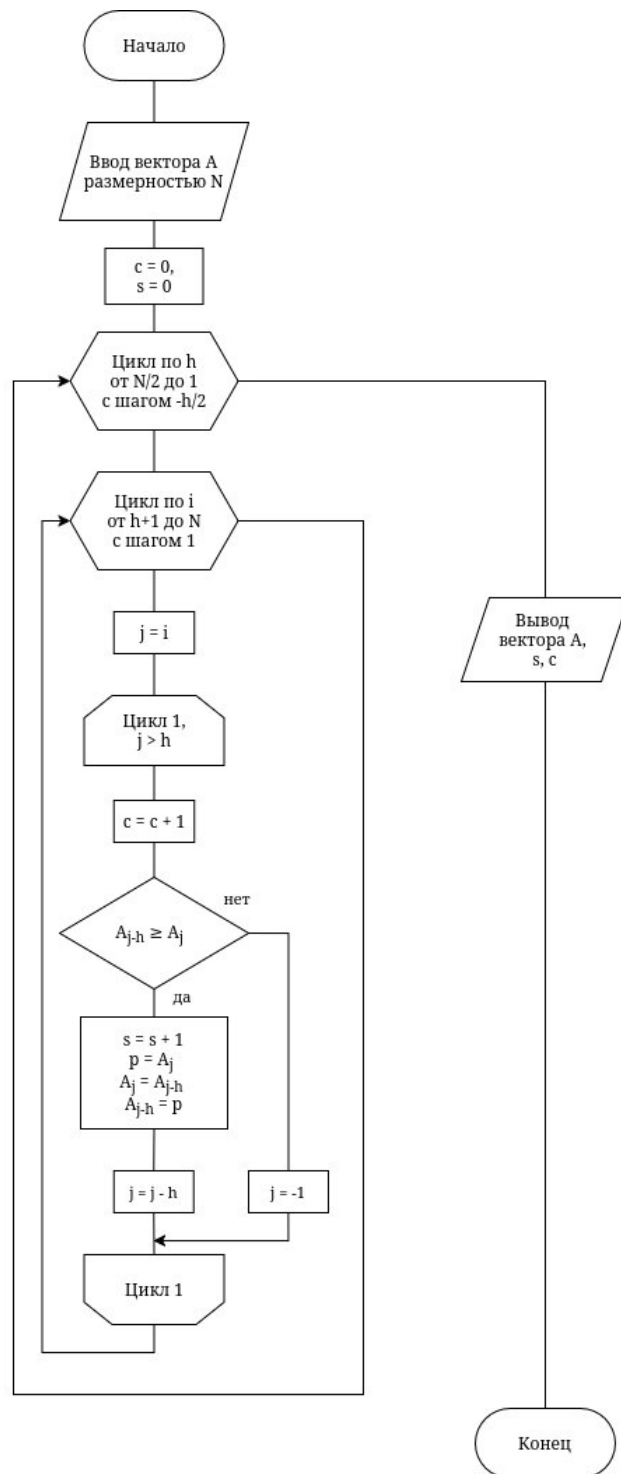


Рисунок 16 – Блок-схема алгоритма сортировки Шелла

Алгоритм быстрой сортировки работает по принципу разделяй и властвуй: выбирается случайный опорный элемент, ставится на свое место так, что элементы слева – меньше него, а справа – больше, затем этот алгоритм применяется для левой и правой части до тех пор, пока длинна сортируемого массива не будет равна одному, ведь в таком случае массив всегда отсортирован. Блок-схема алгоритма быстрой сортировки изображена на рисунке 17.

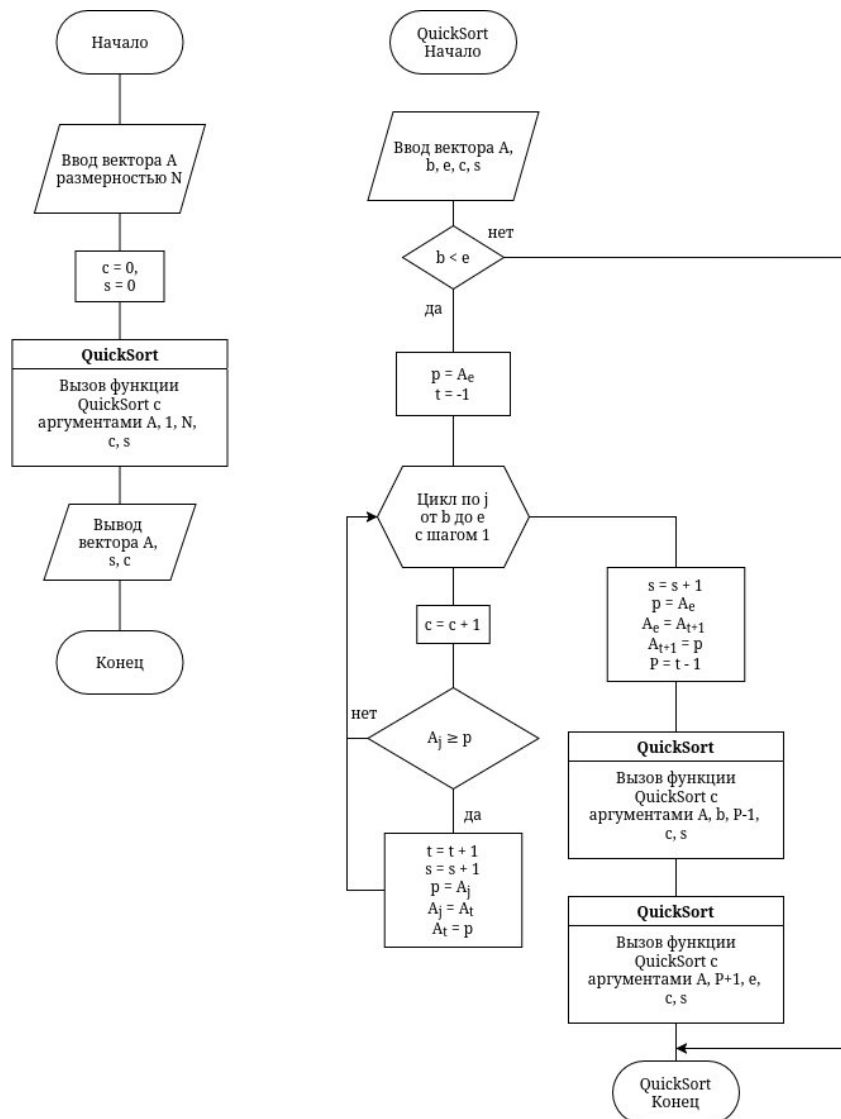


Рисунок 17 – Блок-схема алгоритма быстрой сортировки

3.5 Разработка структуры интерфейсов для пользователя программного комплекса

На рисунке 18 изображена структура интерфейсов для пользователя программного комплекса.



Рисунок 18 – Структура интерфейсов пользователя программного комплекса

Взаимодействуя с интерфейсом, пользователь может сделать несколько действий:

- ввести исходную матрицу вручную или заполнить её случайными значениями,
- ввести исходную матрицу из указанного пользователем файла, предварительно указав её размер,
- сохранить введенную матрицу в указанный пользователем файл,
- выполнить сортировку матрицы и провести анализ алгоритмов сортировки,
- сохранить отсортированную матрицу в указанный пользователем файл,
- выполнить проверку корректности программы (запуск модульных тестов).

При разработке программного комплекса был выбран консольный интерфейс ввиду своей простоты и переносимости: благодаря отсутствию зависимости от графических библиотек программа может запускаться на большем количестве платформ, в том числе на тех, у которых нет графического интерфейса.

При запуске в консоль выводится список доступных действий, где пользователь может выбрать желаемое. Главное меню разработанной программы представлено на рисунке 19.


```
> /home/ilyam0032/Documents/КП/cmake-build-release/ITaP_CP
Выполняется тестирование...
Тестирование пройдено успешно

1) Ввести матрицу
2) Сохранить введенную матрицу в файл
3) Ввести матрицу из файла
4) Отсортировать матрицу и провести анализ
5) Вывести отсортированную матрицу в консоль
6) Сохранить отсортированную матрицу в файл
7) Провести тестирование программы
0) Выход

Выберите соответствующий пункт меню: █
```

Рисунок 19 – Главное меню разработанной программы

3.6 Описание структур данных и алгоритмов

Для того, чтобы программа могла работать с матрицей, её нужно сохранить в оперативной памяти. Это так же применимо ко всем используемым значениям в программе. Поскольку C++ статически типизированный язык, типы данных необходимо явно указывать при объявлении переменных. В таблице 2 указаны типы данных всех используемых переменных в программе.

Исходные данные представляют из себя несортированную матрицу, результатом является отсортированная матрица и сравнительная таблица алгоритмов сортировки.

Таблица 2 – Типы данных переменных, используемых в программе

Название	Тип	Место использования	Описание
entered_matrix, sorted_matrix	optional <vector <vector <int> > >	функция main_menu	Переменные для хранения исходной и отсортированной матрицы. Тип optional позволяет явно определить отсутствие и провести проверки в необходимых местах. Сама же матрица представлена в виде вектора (динамического массива) строк, где каждая строка представляет из себя вектор целых чисел
continue_cycle, input_again	bool	функция main_menu	Флаги продолжить выполнение программы и потребовать ввод снова, участвуют в логике главного меню
rows, columns	int	функции enter_matrix, enter_matrix_from_file	Количество строк и столбцов вводимой матрицы, дополнительно осуществляется проверка на натуральность
decision	bool	функция enter_matrix	Ответ пользователя заполнить ли матрицу случайными значениями
file	ifstream/ ofstream	Функции enter_matrix_from_file/ save_matrix_to_file, open_input_file/ open_output_file	Поток ввода/вывода файла, указанного пользователем
path	string	Функции open_input_file, open_output_file	Путь к файлу, указанный пользователем

3.7 Описание структуры программы

Исходный код программы был разбит на модули. В таблице 3 указаны модули, содержащиеся в них функции и их описание.

Таблица 3 – Модульная структура программы

Модуль	Функция	Описание
Main	main	Точка входа программы
File interface	open_output_file	Функция просит пользователя указать файл, затем возвращает поток вывода для этого файла
	open_input_file	Функция просит пользователя указать файл, затем возвращает поток ввода для этого файла
Console interface	main_menu	Главное консольное меню программы
	enter_matrix_from_file	Функция вызывает open_input_file, затем берет значения оттуда для исходной матрицы из полученного потока
	save_matrix_to_file	Функция вызывает open_output_file, затем записывает переданную в аргументе матрицу в полученный поток
Utils	read_int	Безопасно считать целое число из консольного ввода
	read_natural	Безопасно считать натуральное число из консольного ввода
	read_decision	Считать решение пользователя (да/нет)
	random_int	Получить случайное целое число
	print_matrix	Получает в аргументах матрицу и поток вывода, выводит матрицу в поток
Unit tests	make_tests	Выполнить модульное тестирование программы
Sorting algorithms	sort_matrix	Получает в аргументах матрицу и ссылку на объект ISort, с помощью последнего сортирует матрицу

Наибольший интерес в данной работе представляет модуль sort_matrix. В нем есть всего одна функция, а остальное содержание – классы сортировки. Структура наследования представлена на рисунке 20, описание классов представлена в таблице 4.

Таблица 4 – Описание классов модуля Sorting algorithms

Класс	Член класса	Модификатор доступа	Описание
ISort	comparison	protected	Поле типа unsigned long long. Количество сравнений, произведенных в процессе сортировки
	permutations	protected	Поле типа long long. Количество перестановок, произведенных в процессе сортировки
	get_comparisons	public	Метод, возвращающий количество сравнений
	get_permutations	public	Метод, возвращающий количество сравнений
	Sort	public	Чисто виртуальный метод, принимает в качестве аргумента матрицу, затем сортирует её
BubbleSort	Sort	public	Реализации виртуального метода родительского класса
SelectionSort			
InsertionSort			
ShallSort			
QuickSort			
	Sort	private	Вспомогательный метод с получением индексов начала и конца сортируемого участка, необходим для корректной реализации алгоритма

Тип данных unsigned long long представляет из себя целое число размером 8 байт, обеспечивает диапазон хранимых значений от 0 до 18 446 744 073 709 551 615, что позволяет сохранять корректное количество перестановок и сравнений при сортировке матриц большого размера (содержащих больше миллиона значений), избегая переполнения. Без знаковый тип был выбран, потому что количество совершенных операций не может быть отрицательным.

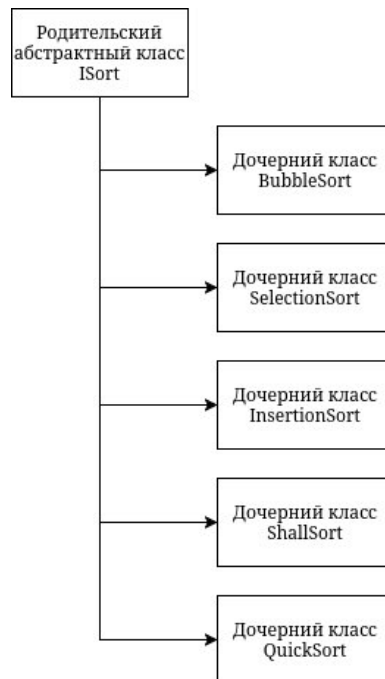


Рисунок 20 – Структура наследования классов сортировки

3.8 Тестирование программного комплекса

Осуществим тестирование программ на матрице размерности 1×5 . Поскольку сортировка строки программы не зависит от других строк, можно предполагать, что если отсортировав матрицу из одной строки результат получится корректным, то и для матрицы, состоящей из более чем одной строки результат программы будет корректным.

Запишем значения матрицы в файл `unsorted_matrix.txt` и введем её в программу, используя интерфейс взаимодействия с файлами. Для корректного чтения матрицы программой, значения необходимо записывать разделяя их одним или несколькими пробелами. На рисунке 21 показано содержимое файла, из которого будут взяты значения матрицы. На рисунке 22 продемонстрирован процесс ввода матрицы из файла.

1	374	-798	906	861	54
2					

Рисунок 21 – Значения матрицы, записанной в файл `unsorted_matrix.txt`

- 1) Ввести матрицу
- 2) Сохранить введенную матрицу в файл
- 3) Ввести матрицу из файла
- 4) Отсортировать матрицу и провести анализ
- 5) Вывести отсортированную матрицу в консоль
- 6) Сохранить отсортированную матрицу в файл
- 7) Провести тестирование программы
- 0) Выход

Выберите соответствующий пункт меню: 3
 Перед получением матрицы из файла укажите её размерность
 Введите количество строк M
 1
 Введите количество столбцов N
 5
 Введите путь к файлу для чтения:
 unsorted_matrix.txt
 Ввод успешно завершён!

Рисунок 22 – Процесс ввода матрицы из файла

Перед проверкой программы предскажем, какой результат она выдаст. Для этого отсортируем введенную матрицу используя каждый алгоритм.

Выберем первую и единственную строку матрицы: 374, -798, 906, 861, 54. Выберем четные элементы для дальнейшей их сортировки: 374, -798, 906, 54.

Отсортируем данную последовательность пузырьковым алгоритмом, для этого выполним шаги:

- 1) сравним 374 и -798 => 374 больше, переставляем их, получаем последовательность: -798, 374, 906, 54
- 2) сравним 374 и 906 => 374 меньше,
- 3) сравним 906 и 54 => 906 больше, переставляем их, получаем -798, 374, 54, 906,
- 4) сравним -798 и 374 => -798 меньше,
- 5) сравним 374 и 54 => 374 больше, переставляем: -798, 54, 374, 906,
- 6) сравним 374 и 906 => 374 меньше,
- 7) сравним -798 и 54 => -798 меньше,
- 8) сравним 54 и 374 => 54 меньше,
- 9) сравним 374 и 906 => 374 меньше, пройдя по всем элементам мы не совершили перестановок, значит последовательность отсортирована.

Было совершено 9 операций сравнения и 3 операции перестановки.

Отсортируем последовательность алгоритмом выбора:

- 1) выберем первый элемент 374,
- 2) сравним его со следующим элементом: 374 и -798 => -798 меньше, теперь выбранный элемент -798,
- 3) сравним со следующим: -798 и 906 => 906 больше,
- 4) сравним со следующим: -798 и 54 => 54 больше,
- 5) проверяем не является ли выбранный первым элементом: сравниваем номер выбранного (2) с 1 => нет, меняем местами выбранный элемент и первый: -798, 374, 906, 54,

6) выбираем второй элемент: 374,
 7) сравниваем с третьим: 374 и 906 $\Rightarrow$ 374 меньше,
 8) сравниваем с четвертым: 374 и 54 $\Rightarrow$ 374 больше, выбираем 54,
 9) сравниваем 2 и 4 на неравенство, меняем их местами: -798, 54, 906, 374,
 10) выбираем третий элемент: 906,
 11) сравниваем с четвертым: 906 и 374 $\Rightarrow$ 906 больше, выбираем 54,
 12) сравниваем 4 и 3 на неравенство, меняем их местами: -798, 54, 374, 906; дошли до конца последовательности, последовательность отсортирована.

Было совершено 9 сравнений и 3 перестановки.

Отсортируем последовательность алгоритмом вставки:

1) начинаем со второго элемента, сравниваем с первым: -798 и 374 $\Rightarrow$ -798 меньше, меняем местами: -798, 374, 906, 54,
 2) сравним третий с предыдущим: 906 и 374 $\Rightarrow$ 906 больше,
 3) сравним четвертый с предыдущим: 54 и 906 $\Rightarrow$ 54 меньше, меняем местами: -798, 374, 54, 906,
 4) сравниваем с предыдущим: 54 и 374 $\Rightarrow$ 54 меньше, меняем местами: -798, 54, 374, 906,
 5) сравниваем с предыдущим: 54 и -798 $\Rightarrow$ 54 больше; дошли до конца, последовательность отсортирована.

Было совершено 5 сравнений и 3 перестановки.

Отсортируем последовательность алгоритмом Шелла:

1) выберем h равным половине длины последовательности: 2,
 2) выберем $h + 1 = 3$ -й элемент, сравним его с $3 - h = 1$ -м элементом: 906 и 374 $\Rightarrow$ 906 больше,
 3) сравним следующие соответственные элементы: 54 и -798 $\Rightarrow$ 54 больше,
 4) выберем h равным половине текущего: 1,
 5) сравним $h + 1 = 2$ -й и $2 - h = 1$ -й элементы: -798 и 374 $\Rightarrow$ -798 меньше, меняем их местами: -798, 374, 906, 54,
 6) сравним следующие за ними элементы: 906 и 374 $\Rightarrow$ 906 больше,
 7) сравним следующие за ними элементы: 54 и 906 $\Rightarrow$ 54 меньше, меняем их местами: -798, 374, 54, 906,
 8) сравним предыдущие перед ними: 54 и 374 $\Rightarrow$ 54 меньше, меняем их местами: -798, 54, 374, 906,
 9) сравним предыдущие перед ними: 54 и -798 $\Rightarrow$ 54 больше,
 10) берем h равным половине текущего: 0 в целочисленной арифметике, последовательность отсортирована.

Было совершено 7 сравнений и 3 перестановки.

Отсортируем последовательность алгоритмом быстрой сортировки:

1) начальный элемент первый, конечный элемент – четвертый, выберем опорным конечный элемент 54, номер последнего элемента, меньшего опорного (далее ПМО) равен 0 (пока все элементы считаются большим опорного);

2) сравним первый элемент с опорным: 374 и 54 $\Rightarrow$ 374 больше;

3) сравним следующий элемент с опорным: -798 и 54 $\Rightarrow$ -798 меньше, увеличиваем номер ПМО на 1 и переставляем текущий элемент с этим: -798, 374, 906, 54;

4) сравним следующий элемент с опорным: 906 и 54 $\Rightarrow$ 906 больше;

5) переставляем опорный элемент со следующим за ПМО: -798, 54, 906, 374;

6) сортируем элементы меньше опорного: -798 состоит из одного числа, значит отсортировано;

7) сортируем элементы больше опорного: 906, 374;

а) выбираем 374 опорным, ПМО = 0;

б) сравниваем первый элемент с опорным: 906 и 374 $\Rightarrow$ 906 больше;

в) меняем опорный со следующим за ПМО: 374, 906;

г) сортируем элементы до опорного: их нет;

д) сортируем элементы после опорного: 906 – отсортировано;

8) результат: 374, 906;

9) соединяем элементы до опорного, опорный и после опорного: -798, 54, 374, 906, последовательность отсортирована.

Было произведено 4 сравнения и 3 перестановки.

В результате применения алгоритмов сортировки к последовательности четных чисел получалась последовательность: -798, 54, 374, 906. Теперь вернем четные числа на соответственные места в строке матрицы: -798, 54, 376, 861, 906.

Предполагаемый результат работы программы известен, теперь посмотрим, что выдала программа. На рисунке 23 представлен результат работы программы.

Исходная матрица:
 374 -798 906 861 54

Матрица, отсортированная пузырьковой сортировкой:
 -798 54 374 861 906

Количество сравнений элементов: 9
 Количество перестановок элементов: 3
 Матрица, отсортированная сортировкой выбором:
 -798 54 374 861 906

Количество сравнений элементов: 9
 Количество перестановок элементов: 3
 Матрица, отсортированная сортировкой вставкой:
 -798 54 374 861 906

Количество сравнений элементов: 5
 Количество перестановок элементов: 3
 Матрица, отсортированная сортировкой Шелла:
 -798 54 374 861 906

Количество сравнений элементов: 7
 Количество перестановок элементов: 3
 Матрица, отсортированная быстрой сортировкой:
 -798 54 374 861 906

Количество сравнений элементов: 4
 Количество перестановок элементов: 3

Сравнение методов сортировки:

Алгоритм сортировки	Количество сравнений	Количество перестановок	Суммарное количество операций
Пузырьковая сортировка	9	3	12
Сортировка выбором	9	3	12
Сортировка вставкой	5	3	8
Сортировка Шелла	7	3	10
Быстрая сортировка	4	3	7

Наиболее эффективной оказалась: быстрая сортировка

Рисунок 23 – Результат работы программы

Как можно видеть, предполагаемый результат совпадает с результатом работы программы – это означает, что программа работает корректно.

3.9 Характеристика программного и аппаратного обеспечения.

На рисунке 24 представлена структура программного обеспечения. Программный комплекс был разработан под операционную систему Arch Linux. Средой разработки является CLion 2025.3.3 .

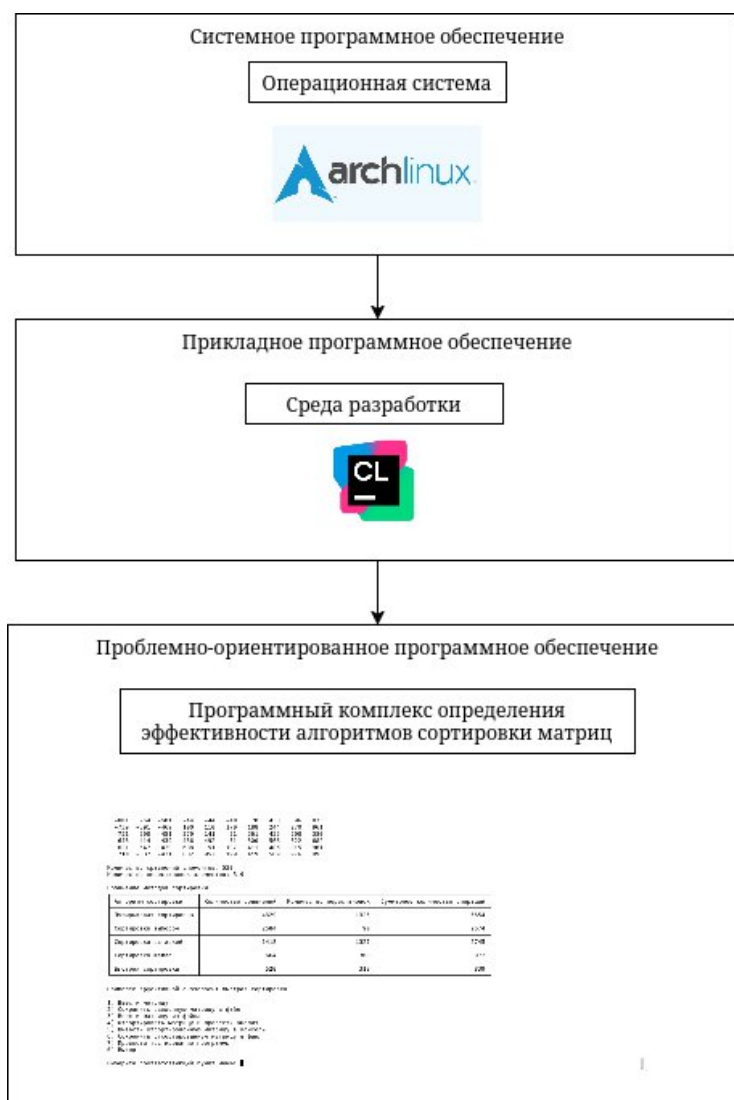


Рисунок 24 – Структура программного обеспечения

В таблице 5 приведена характеристика проблемно-ориентированного программного обеспечения.

Таблица 5 – Характеристика программного обеспечения

Показатель	Значение
Среда разработки	CLion 2025.3.3
Язык программирования	C++
Количество входных переменных	3
Количество внутренних переменных	9
Количество выходных элементов	13
Количество классов, структур	6
Количество функций	12
Размер исполняемого файла, КБ	72,8
Время обработки результатов и визуализации результатов, мс	50

Требования к ЭВМ для нормального функционирования программного комплекса представлены в таблице 6.

Таблица 6 – Минимальные системные требования

Показатель	Значение
Тип ЭВМ	Персональный компьютер
Тактовая частота процессора, ГГц	2
Объем оперативной памяти, ГБ	8
Состав и характеристика периферийных устройств	Клавиатура, дисплей с разрешением 1024×1336 пикселей
Операционная система	Arch Linux
Прикладное программное обеспечение, необходимое для функционирования программного комплекса	CLion 2025.3.3

ВЫВОДЫ ПО ПРОЕКТУ

В результате выполнения курсового проекта разработан программный комплекс, использующий полиморфизм в своей основе и позволяющий решать задачу сортировки матрицы методом выравнивания, а также проводить сравнительный анализ различных методов сортировки.

Проведён анализ предметной области: изучены основные понятия матричных вычислений, виды сортировки матриц, выполнено сравнение систем-аналогов (Excel, Mathcad, Scilab), показавшее необходимость разработки собственного комплекса для демонстрации полиморфизма.

Составлено формализованное описание, в котором определены исходные данные (матрица A размером $N \times M$), выходные данные (матрица A' с упорядоченными четными числами по строкам и упорядоченными нечетными числами по столбцам).

Поставлена задача обработки информации, для заданной матрицы $A_{N \times M}$ подобрать такие V , что, либо s , либо c будут минимальными, а также вывести отсортированную матрицу $A'_{N \times M}$.

Разработана функциональная структура, включающая модули вывода результатов, реализации сортировок и подсчёта статистической информации, сохранения данных, работы с файлами, пользовательского интерфейса.

Разработаны пять алгоритмов сортировки: пузырьковая, отбором, вставками, Шелла и быстрая, и отдельный алгоритм сортировки матрицы упорядочиванием каждой строки матрицы по возрастанию четных чисел.

Разработан консольный интерфейс, включающий подсистемы ввода, запуска сортировок и вывода результатов, сохранения исходной матрицы в файл, запуска модульных тестов.

Проведено тестирование корректности работы алгоритмов и ввода исходных данных из файлов.

Разработанный программный комплекс наглядно демонстрирует принципы полиморфизма, позволяет выполнять сортировку динамических матриц различными методами и проводить сравнительный анализ эффективности алгоритмов по числу сравнений и перестановок.

Пути дальнейшего развития: обработка трехмерных массивов данных, использование параллельных вычислений, использование GPU вычислений, использование асинхронных вычислений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Чистякова, Т. Б. Программирование на языках высокого уровня. Базовый курс / Т. Б. Чистякова, Р. В. Антипин, И. В. Новожилова. – Санкт-Петербург : СПбГТИ(ТУ), 2008. – 227 с. – ISBN 978-5-91884-017-7.
- 2 Павловская, Т. А. С/С++. Программирование на языке высокого уровня / Т. А. Павловская. – Санкт-Петербург: Питер, 2021. – 461 с. – ISBN 978-5-4461-3916-3.
- 3 Лафоре, Р. Объектно-ориентированное программирование в С++ / Р. Лафоре. – 4-е изд. – Санкт-Петербург : Питер, 2022. – 928 с. – ISBN 978-5-4461-0927-2.
- 4 Иванова, Г. С. Объектно-ориентированное программирование / Г. С. Иванова, Т. Н. Ничушкина, Е. К. Пугачев. – Москва : МГТУ, 2007. – 366 с. – ISBN 978-5-7038-2775-8.
- 5 Гутгарц, Р. Д. Проектирование автоматизированных систем обработки информации и управления : учебник для вузов / Р. Д. Гутгарц. – 2-е изд., перераб. и доп. – Москва : Издательство Юрайт, 2026. – 351 с. – (Высшее образование). – ISBN 978-5-534-15761-1.
- 6 Алгоритмы. Построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн. – 3-е изд. – Москва : Вильямс, 2013. – 1328 с. – ISBN 978-5-8459-1794-6.
- 7 Седжвик, Р. Фундаментальные алгоритмы на С++. Анализ. Структуры данных. Сортировка. Поиск. / Р. Седжвик – Киев : ДиаСофт, 2001. – 688 с. – ISBN 966-7393-89-5.

Приложение А

```
// main.cpp
// Точка входа
// Студент группы 4503, Илья М, 2026

#include "unit_tests.h"
#include "console_interface.h"

#include <ctime>
#include <iostream>

int main() {
    std::srand(std::time(nullptr));
    if (make_tests()) {
        std::cout << "Тестирование пройдено успешно" << std::endl;
        main_menu();
        return EXIT_SUCCESS;
    }
    return EXIT_SUCCESS;
}

// unit_tests.h
// Заголовочный файл модульного тестирования
// Студент группы 4503, Илья М, 2026

#ifndef ITAP_CP_UNIT_TESTS_H
#define ITAP_CP_UNIT_TESTS_H

bool make_tests();

#endif // ITAP_CP_UNIT_TESTS_H

// unit_tests.cpp
// Файл модульного тестирования
// Студент группы 4503, Илья М, 2026
```

```

#include <limits.h>

#include <vector>

#include <iostream>

#include "sorting_algorithms.h"

bool sorted(const std::vector<int>& v) {
    int least = -INT_MAX;
    for (auto& i : v) {
        if (least > i) {
            return false;
        }
        least = i;
    }
    return true;
}

bool make_tests() {
    bool success = true;

    std::cout << "Выполняется тестирование..." << std::endl;
    if (not sorted({1,2,3})) {
        std::cout << "Тест 1 не пройден." << std::endl;
        success = false;
    }

    if (sorted({3,2,1})) {
        std::cout << "Тест 2 не пройден." << std::endl;
        success = false;
    }

    if (not sorted({1,2,2,3})) {
        std::cout << "Тест 3 не пройден." << std::endl;
        success = false;
    }
}

```

```
std::vector<int> unsorted_vector = {2, 5, 10, -1, 4, 4};

BubbleSort bubble_sort;
auto bubble_sorted_vector = unsorted_vector;
bubble_sort.Sort(bubble_sorted_vector);
if (not sorted(bubble_sorted_vector)) {
    std::cout << "Тест 4 не пройден." << std::endl;
    success = false;
}

SelectionSort selection_sort;
auto selection_sorted_vector = unsorted_vector;
selection_sort.Sort(selection_sorted_vector);
if (not sorted(selection_sorted_vector)) {
    std::cout << "Тест 5 не пройден." << std::endl;
    success = false;
}

InsertionSort insertion_sort;
auto insertion_sorted_vector = unsorted_vector;
insertion_sort.Sort(insertion_sorted_vector);
if (not sorted(insertion_sorted_vector)) {
    std::cout << "Тест 6 не пройден." << std::endl;
    success = false;
}

ShallSort shall_sort;
auto shall_sorted_vector = unsorted_vector;
shall_sort.Sort(shall_sorted_vector);
if (not sorted(shall_sorted_vector)) {
    std::cout << "Тест 7 не пройден." << std::endl;
    success = false;
}
```



```

QuickSort quick_sort;

auto quick_sorted_vector = unsorted_vector;

quick_sort.Sort(quick_sorted_vector);

if (not sorted(quick_sorted_vector)) {
    std::cout << "Тест 8 не пройден." << std::endl;
    success = false;
}

return success;
}

// console_interface.h
// Консольный интерфейс программы
// Студент группы 4503, Илья М, 2026

#ifndef ITAP_TESTS_MAIN_MENU_H
#define ITAP_TESTS_MAIN_MENU_H

#include <vector>

enum main_menu_options {
    OPTION_EXIT = 0,
    OPTION_ENTER_MATRIX = 1,
    OPTION_SAVE_ENTER_MATRIX = 2,
    OPTION_ENTER_MATRIX_FROM_FILE = 3,
    OPTION_MAKE_ANALYSE = 4,
    OPTION_PRINT_SORTED_MATRIX = 5,
    OPTION_SAVE_SORTED_MATRIX_TO_FILE = 6,
    OPTION_START_UNIT_TESTS = 7,
};

void main_menu();

void enter_matrix(std::vector<std::vector<int>>& matrix);

void enter_matrix_from_file(std::vector<std::vector<int>>& matrix);

void make_analyse_on_matrix(const std::vector<std::vector<int>>& matrix,
std::vector<std::vector<int>>& result_matrix);

void save_matrix_to_file(const std::vector<std::vector<int>>& sorted_matrix);

```

```

#endif // ITAP_TESTS_MAIN_MENU_H

// console_interface.cpp
// Консольный интерфейс программы
// Студент группы 4503, Илья М, 2026

#include "console_interface.h"

#include "utils.h"
#include "sorting_algorithms.h"
#include "file_interface.h"
#include "unit_tests.h"

#include <algorithm>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <optional>

#define PERM_COLUMN_WITH 23
#define COMP_COLUMN_WITH 20
#define SUMM_COLUMN_WITH 29

void main_menu() {
    std::optional<std::vector<std::vector<int>>>> entered_matrix;
    std::optional<std::vector<std::vector<int>>>> sorted_matrix;

    bool continue_cycle = true;
    do {
        std::cout << std::endl;
        std::cout << "1) Ввести матрицу" << std::endl;

```

```

std::cout << "2) Сохранить введенную матрицу в файл" << std::endl;

std::cout << "3) Ввести матрицу из файла" << std::endl;

std::cout << "4) Отсортировать матрицу и провести анализ" <<
std::endl;

std::cout << "5) Вывести отсортированную матрицу в консоль" <<
std::endl;

std::cout << "6) Сохранить отсортированную матрицу в файл" <<
std::endl;

std::cout << "7) Провести тестирование программы" << std::endl;

std::cout << "0) Выход" << std::endl;

std::cout << std::endl;

std::cout << "Выберите соответствующий пункт меню: ";

bool input_again = false;
do {
    input_again = false;
    switch (read_int()) {
    case OPTION_ENTER_MATRIX:
        if (!entered_matrix) {
            entered_matrix.emplace();
        }
        enter_matrix(entered_matrix.value());
        break;

    case OPTION_SAVE_ENTER_MATRIX:
        if (!entered_matrix) {
            std::cout << "Исходная матрица пуста." << std::endl;
            break;
        }
        save_matrix_to_file(entered_matrix.value());
        break;

    case OPTION_ENTER_MATRIX_FROM_FILE:
        if (!entered_matrix) {
            entered_matrix.emplace();
        }

```

```

        enter_matrix_from_file(entered_matrix.value());
        break;

case OPTION_MAKE_ANALYSE:
    if (!entered_matrix) {
        std::cout << "Исходная матрица пуста." << std::endl;
        break;
    }
    if (!sorted_matrix) {
        sorted_matrix.emplace();
    }
    make_analyse_on_matrix(entered_matrix.value(),
sorted_matrix.value());
    break;

case OPTION_PRINT_SORTED_MATRIX:
    if (!sorted_matrix) {
        std::cout << "Сортировка матрицы ещё не была
произведена";
        break;
    }
    std::cout << "Отсортированная матрица" << std::endl;
    print_matrix(sorted_matrix.value(), std::cout);
    // print_sorted_matrix(sorted_matrix);
    break;

case OPTION_SAVE_SORTED_MATRIX_TO_FILE:
    if (!sorted_matrix) {
        std::cout << "Сортировка матрицы ещё не была
произведена";
        break;
    }
    save_matrix_to_file(sorted_matrix.value());
    break;

case OPTION_START_UNIT_TESTS:

```

```

        if (make_tests()) {
            std::cout << "Тестирование пройдено успешно" <<
std::endl;

        }

        break;

    case OPTION_EXIT:
        continue_cycle = false;
        break;

    default:
        std::cout << "Нет такого пункта меню. Попробуйте ещё раз: ";
        input_again = true;
    }
} while (input_again);
} while (continue_cycle);
}

void enter_matrix(std::vector<std::vector<int>>& matrix) {
    // 1) Получить количество столбцов
    // 2) получить количество строк
    // 3) заполнить случайными значениями?
    // 4) нет - заполнить вручную

    std::cout << "Введите количество строк M" << std::endl;

    int rows = read_natural();

    std::cout << "Введите количество столбцов N" << std::endl;

    int columns = read_natural();

    matrix.assign(rows, std::vector<int>(columns, 0));

    std::cout << "Ввести " << rows*columns<< " значений вручную? [Y/N]" <<
std::endl;

    bool decision = read_decision();

```

```

    for (size_t row = 0; row < rows; row++) {
        for (size_t column = 0; column < columns; column++) {
            if (decision) {
                std::cout << "Введите значение [" << row+1 << "][" <<
column+1 << "]" : " << std::endl;

                matrix[row][column] = read_int();
            } else {
                matrix[row][column] = random_int();
            }
        }
    }

    std::cout << "Ввод успешно завершен!" << std::endl;
}

void enter_matrix_from_file(std::vector<std::vector<int>>& matrix) {
    std::cout << "Перед получением матрицы из файла укажите её размерность"
<< std::endl;

    std::cout << "Введите количество строк M" << std::endl;

    int rows = read_natural();

    std::cout << "Введите количество столбцов N" << std::endl;

    int columns = read_natural();

    matrix.assign(rows, std::vector<int>(columns, 0));

    std::ifstream file = open_input_file();
    bool insufficient_values = false;
    for (size_t row = 0; row < rows; row++) {
        for (size_t column = 0; column < columns; column++) {
            int value = 0;

            if (insufficient_values) {
                matrix[row][column] = 0;
                continue;
            }

            if (!(file >> value)) {
                if (file.eof()) {

```

```

        std::cout << "Значений в файле меньше чем требуется для
заполнения матрицы. Недостающие элементы были заполнены нулями." <<
std::endl;

        insufficient_values = true;
    }

    else {

        std::cout << "Ошибка чтения в позиции " << row * columns
+ column << ". Матрица была заполнена частично" << std::endl;

        return;

    }

    }

    matrix[row][column] = value;

}

}

std::cout << "Ввод успешно завершен!" << std::endl;

file.close();

}

void make_analyse_on_matrix(const std::vector<std::vector<int>>& matrix,
std::vector<std::vector<int>>& result_matrix) {

    // вывести исходную матрицу

    std::cout << "Исходная матрица: " << std::endl;

    print_matrix(matrix, std::cout);

    std::cout << std::endl;

    // 1) скопировать, отсортировать пузырьковой сортировкой
    // вывести отсортированную, вывести статистику по методу

    auto bubble_sorted_matrix = matrix;

    BubbleSort bubble_sort;

    sort_matrix(bubble_sorted_matrix, bubble_sort);

    std::cout << "Матрица, отсортированная пузырьковой сортировкой: " <<
std::endl;

    print_matrix(bubble_sorted_matrix, std::cout);

    std::cout << std::endl;

    std::cout << "Количество сравнений элементов: " <<
bubble_sort.get_comparisons() << std::endl;

```

```

    std::cout << "Количество перестановок элементов: " <<
    bubble_sort.get_permutations() << std::endl;

    // 2) скопировать, отсортировать сортировкой отбором

    // вывести отсортированную, вывести статистику по методу

    auto selection_sorted_matrix = matrix;

    SelectionSort selection_sort;

    sort_matrix(selection_sorted_matrix, selection_sort);

    std::cout << "Матрица, отсортированная сортировкой выбором: " <<
    std::endl;

    print_matrix(selection_sorted_matrix, std::cout);

    std::cout << std::endl;

    std::cout << "Количество сравнений элементов: " <<
    selection_sort.get_comparisons() << std::endl;

    std::cout << "Количество перестановок элементов: " <<
    selection_sort.get_permutations() << std::endl;

    if (bubble_sorted_matrix != selection_sorted_matrix) {

        std::cout << "Ошибка, матрицы отсортированные пузырьком и выбором не
        совпадают!" << std::endl;

        // return;

    }

    // 3) скопировать, отсортировать сортировкой вставкой

    // вывести отсортированную, вывести статистику по методу

    auto insertion_sorted_matrix = matrix;

    InsertionSort insertion_sort;

    sort_matrix(insertion_sorted_matrix, insertion_sort);

    std::cout << "Матрица, отсортированная сортировкой вставкой: " <<
    std::endl;

    print_matrix(insertion_sorted_matrix, std::cout);

    std::cout << std::endl;

    std::cout << "Количество сравнений элементов: " <<
    insertion_sort.get_comparisons() << std::endl;

    std::cout << "Количество перестановок элементов: " <<
    insertion_sort.get_permutations() << std::endl;

```



```

    if (selection_sorted_matrix!= insertion_sorted_matrix) {
        std::cout << "Ошибка, матрицы отсортированные выбором и вставкой не
совпадают!" << std::endl;

        // return;
    }

    // 4) скопировать, отсортировать сортировкой Шелла
    // вывести отсортированную, вывести статистику по методу

    auto shall_sorted_matrix = matrix;
    ShallSort shall_sort;
    sort_matrix(shall_sorted_matrix, shall_sort);
    std::cout << "Матрица, отсортированная сортировкой Шелла: " << std::endl;
    print_matrix(shall_sorted_matrix, std::cout);
    std::cout << std::endl;

    std::cout << "Количество сравнений элементов: " <<
shall_sort.get_comparisons() << std::endl;

    std::cout << "Количество перестановок элементов: " <<
shall_sort.get_permutations() << std::endl;

    if (insertion_sorted_matrix!= shall_sorted_matrix) {
        std::cout << "Ошибка, матрицы отсортированные вставкой и Шеллом не
совпадают!" << std::endl;

        // return;
    }

    // 5) скопировать, отсортировать быстрой сортировкой
    // вывести отсортированную, вывести статистику по методу

    auto quick_sorted_matrix = matrix;
    QuickSort quick_sort;
    sort_matrix(quick_sorted_matrix, quick_sort);
    std::cout << "Матрица, отсортированная быстрой сортировкой: " <<
std::endl;
    print_matrix(quick_sorted_matrix, std::cout);

```

```

std::cout << std::endl;

std::cout << "Количество сравнений элементов: " <<
quick_sort.get_comparisons() << std::endl;

std::cout << "Количество перестановок элементов: " <<
quick_sort.get_permutations() << std::endl;

std::cout << std::endl;

if (shall_sorted_matrix!= quick_sorted_matrix) {

    std::cout << "Ошибка, матрицы отсортированные Шеллом и быстрой не
совпадают!" << std::endl;

    // return;

}

std::cout << "Сравнение методов сортировки: " << std::endl;

std::cout <<
" |-----| " << std::endl;

std::cout << " | Алгоритм сортировки | Количество сравнений |
Количество перестановок | Суммарное количество операций | " << std::endl;

std::cout <<
" |-----| " << std::endl;

std::cout << " | Пузырьковая сортировка | "
<< std::setw(COMP_COLUMN_WITH) << bubble_sort.get_comparisons() << " | "
<< std::setw(PERM_COLUMN_WITH) << bubble_sort.get_permutations() << " | "
<< std::setw(SUMM_COLUMN_WITH) << bubble_sort.get_permutations() +
bubble_sort.get_comparisons() << " | " << std::endl;

std::cout <<
" |-----| " << std::endl;

std::cout << " | Сортировка выбором | "
<< std::setw(COMP_COLUMN_WITH) << selection_sort.get_comparisons() << " | "
<< std::setw(PERM_COLUMN_WITH) << selection_sort.get_permutations() << " | "
<< std::setw(SUMM_COLUMN_WITH) << selection_sort.get_permutations() +
selection_sort.get_comparisons() << " | " << std::endl;

std::cout <<
" |-----| " << std::endl;

std::cout << " | Сортировка вставкой | "

```

```

    << std::setw(COMP_COLUMN_WITH) << insertion_sort.get_comparisons() << " |
"

    << std::setw(PERM_COLUMN_WITH) << insertion_sort.get_permutations() << "
| "

    << std::setw(SUMM_COLUMN_WITH) << insertion_sort.get_permutations() +
insertion_sort.get_comparisons() << " |" << std::endl;

    std::cout <<
" |-----|-----|-----|
|-----|" << std::endl;

    std::cout << " | Сортировка Шелла | "

    << std::setw(COMP_COLUMN_WITH) << shall_sort.get_comparisons() << " | "

    << std::setw(PERM_COLUMN_WITH) << shall_sort.get_permutations() << " | "

    << std::setw(SUMM_COLUMN_WITH) << shall_sort.get_permutations() +
shall_sort.get_comparisons() << " |" << std::endl;

    std::cout <<
" |-----|-----|-----|
|-----|" << std::endl;

    std::cout << " | Быстрая сортировка | "

    << std::setw(COMP_COLUMN_WITH) << quick_sort.get_comparisons() << " | "

    << std::setw(PERM_COLUMN_WITH) << quick_sort.get_permutations() << " | "

    << std::setw(SUMM_COLUMN_WITH) << quick_sort.get_permutations() +
quick_sort.get_comparisons() << " |" << std::endl;

    std::cout <<
" |-----|-----|-----|
|-----|" << std::endl;

    std::cout << std::endl;

    std::vector<std::pair<std::string, unsigned long long>> stats = {
        {"пузырьковая сортировка", bubble_sort.get_permutations() +
bubble_sort.get_comparisons()},
        {"сортировка выбором", selection_sort.get_permutations() +
selection_sort.get_comparisons()},
        {"сортировка вставкой", insertion_sort.get_permutations() +
insertion_sort.get_comparisons()},
        {"сортировка Шелла", shall_sort.get_permutations() +
shall_sort.get_comparisons()},
        {"быстрая сортировка", quick_sort.get_permutations() +
quick_sort.get_comparisons() }

```

```

};

auto min_element_iterator = std::min_element(
    stats.begin(), stats.end(),
    [](auto const &a, auto const &b){ return a.second < b.second; }
);

unsigned long long least_ops = min_element_iterator->second;

bool success = true;

if (bubble_sorted_matrix!= selection_sorted_matrix) {
    std::cout << "Ошибка, матрицы отсортированные пузырьком и выбором не совпадают!" << std::endl;
    success = false;
}

if (selection_sorted_matrix!= insertion_sorted_matrix) {
    std::cout << "Ошибка, матрицы отсортированные выбором и вставкой не совпадают!" << std::endl;
    success = false;
}

if (insertion_sorted_matrix!= shall_sorted_matrix) {
    std::cout << "Ошибка, матрицы отсортированные вставкой и Шеллом не совпадают!" << std::endl;
    success = false;
}

if (shall_sorted_matrix!= quick_sorted_matrix) {
    std::cout << "Ошибка, матрицы отсортированные Шеллом и быстрой не совпадают!" << std::endl;
    success = false;
}

if (success) {
    result_matrix = std::move(quick_sorted_matrix);
    std::cout << "Наиболее эффективными оказались: ";
    std::for_each(
        stats.begin(), stats.end(), [&](auto const &x) {
            if (x.second == least_ops) std::cout << x.first << "; ";
        }
    );
}

```

```

        });

    } else {

        std::cout << "В процессе сортировки получились разые матрицы,
отсортированная матрица не была сохранена" << std::endl;

    }

    std::cout << std::endl;
}

void save_matrix_to_file(const std::vector<std::vector<int>>& sorted_matrix)
{
    if (sorted_matrix.empty()) {
        std::cout << "Матрица пуста" << std::endl;
        return;
    }

    auto file = open_output_file();
    print_matrix(sorted_matrix, file);
    std::cout << "Успешно!" << std::endl;
    file.close();
}

// file_interface.h
// Файловый интерфейс программы
// Студент группы 4503, Илья М, 2026

#ifndef ITAP_CP_FILE_INTERFACE_H
#define ITAP_CP_FILE_INTERFACE_H
#include <iosfwd>

std::ofstream open_output_file();
std::ifstream open_input_file();
#endif // ITAP_CP_FILE_INTERFACE_H

// file_interface.cpp

```

```

// Файловый интерфейс программы
// Студент группы 4503, Илья М, 2026

#include "file_interface.h"

#include "utils.h"

#include <filesystem>
#include <fstream>
#include <iosfwd>
#include <iostream>

std::ofstream open_output_file() {
    while (true) {
        std::string path;

        std::cout << "Введите путь к файлу для записи: " << std::endl;
        std::getline(std::cin, path);

        if (std::filesystem::exists(path)) {
            std::cout << "Ввести " << rows*columns<< " значений вручную?
[Y/N]" << std::endl;

            bool decision = read_decision();

            }

            std::ofstream output_file(path, std::ios::out);
            if (output_file.is_open()) {
                return output_file;
            }

            std::cout << "Не удалось открыть файл, попробуйте ещё раз" <<
std::endl;

        }
    }

std::ifstream open_input_file() {
    while (true) {

```

```

        std::string path;

        std::cout << "Введите путь к файлу для чтения: " << std::endl;

        std::getline(std::cin, path);

        std::ifstream input_file(path, std::ios::in);

        if (input_file.is_open()) {

            return input_file;

        }

        std::cout << "Не удалось открыть файл, попробуйте ещё раз" <<
std::endl;

    }

}

// utils.h
// Вспомогательный функции
// Студент группы 4503, Илья М, 2026

#ifndef UTILS_H
#define UTILS_H
#include <memory>
#include <vector>

int read_int();

int read_natural();
bool read_decision();

int random_int();

void print_matrix(const std::vector<std::vector<int>>& matrix, std::ostream&
out);

#endif //UTILS_H

```

```
// utils.cpp
// Вспомогательные функции
// Студент группы 4503, Илья М, 2026

#include "utils.h"

#define RAND_MIN_ (-1000)
#define RAND_MAX_ 1000
#define MATRIX_COLUMN_WITH 6

#include <ctime>
#include <iomanip>
#include <iostream>
#include <string>

int read_int() {
    std::string input;
    bool success = false;
    int result = 0;

    do {
        try {
            std::getline(std::cin, input);
            result = std::stoi(input);
            success = true;
        } catch (std::invalid_argument &) {
            std::cout << "Некорректный ввод. Попробуйте ещё раз: ";
        }
    } while (!success);
}
```



```

    return result;
}

int read_natural() {
    int result = 0;
    bool valid = false;
    do {
        result = read_int();
        valid = (result > 0);
        if (!valid) std::cout << "Полученное число не натуральное. Попробуйте
ещё раз: ";
    } while (!valid);
    return result;
}

do {
    std::string input;
    std::getline(std::cin, input);
    if (input.empty()) {
        std::cout << "Некорректный ввод. Введите Y или N: ";
        continue;
    }

    switch (input[0]) {
        case 'Y':
        case 'y':
            return true;
        case 'N':
        case 'n':
            return false;
        default:
            std::cout << "Некорректный ввод. Введите Y или N: ";
    }
}

```

```

        }
    } while (true);
}

int random_int() {
    return RAND_MIN_ + rand() % (RAND_MAX_ - RAND_MIN_);
}

void print_matrix(const std::vector<std::vector<int>>& matrix, std::ostream&
out) {
    for (auto& line : matrix) {
        for (auto& element: line) {
            out << std::setw(MATRIX_COLUMN_WITH) << element;
        }
        out << std::endl;
    }
}

// sorting_algorithms.h
// Алгоритмы сортировок
// Студент группы 4503, Илья М, 2026

#ifndef ITAP_CP_SORTING_ALGORITHMS_H
#define ITAP_CP_SORTING_ALGORITHMS_H
#include <vector>

class ISort {
protected:
    unsigned long long comparison = 0;
    unsigned long long permutations = 0;

```

```

public:

    virtual ~ISort() = default;

    virtual void Sort(std::vector<int>& array) = 0;


    unsigned long long get_comparisons() const { return comparison; }
    unsigned long long get_permutations() const { return permutations; }
};


class BubbleSort : public ISort {
public:
    void Sort(std::vector<int>& array) override;
};

class SelectionSort : public ISort {
public:
    void Sort(std::vector<int>& array) override;
};

class InsertionSort : public ISort {
public:
    void Sort(std::vector<int>& array) override;
};

class ShallSort : public ISort {
public:
    void Sort(std::vector<int>& array) override;
};

class QuickSort : public ISort {
    void Sort(std::vector<int>& array, int begin, int end);
public:
    void Sort(std::vector<int>& array) override;
};


void sort_matrix(std::vector<std::vector<int>>& matrix, ISort&
sorting_class);


#endif // ITAP_CP_SORTING_ALGORITHMS_H

```

```
// sorting_algorithms.cpp
// Алгоритмы сортировок
// Студент группы 4503, Илья М, 2026

#include "sorting_algorithms.h"

void BubbleSort::Sort(std::vector<int>& array) {
    if (array.size() < 2) {
        return;
    }
    int previous_permutations;

    do {
        previous_permutations = permutations;
        for (int i = 0; i < array.size() - 1; i++) {
            ++comparison;
            if (array[i] > array[i + 1]) {
                ++permutations;
                std::swap(array[i], array[i + 1]);
            }
        }
    } while (permutations > previous_permutations);
}

void SelectionSort::Sort(std::vector<int>& array) {
    if (array.size() < 2) {
        return;
    }

    for (int i = 0; i < array.size() - 1; i++) {
        int selected_element = i;
```

```

        for (int j = i + 1; j < array.size() ; j++) {
            ++comparison;
            if (array[j] < array[selected_element]) {
                selected_element = j;
            }
        }
        ++comparison;
        if (selected_element != i) {
            ++permutations;
            std::swap(array[i], array[selected_element]);
        }
    }
}

```

```

void InsertionSort::Sort(std::vector<int>& array) {
    if (array.size() < 2) {
        return;
    }
    for (int i = 1; i < array.size(); i++) {
        for (int j = i; j > 0; j--) {
            ++comparison;
            if (array[j-1] >= array[j]) {
                ++permutations;
                std::swap(array[j], array[j-1]);
            } else {
                break;
            }
        }
    }
}

```

```

void ShallSort::Sort(std::vector<int> &array) {
    if (array.size() < 2) {
        return;
    }
}

```

```

    }

    for (int h = array.size()/2; h > 0; h/=2) {
        for (int i = h; i < array.size(); i++) {
            // int temp = array[i];

            for (int j = i; j >= h; j -= h) {
                ++comparison;

                if (array[j-h] > array[j]) {
                    ++permutations;

                    std::swap(array[j], array[j-h]);
                } else {
                    break;
                }
            }
        }
    }
}

void QuickSort::Sort(std::vector<int>& array) {
    if (array.size() < 2) {
        return;
    }

    Sort(array, 0, array.size() - 1);
}

void QuickSort::Sort(std::vector<int>& array, const int begin, const int end)
{
    if (begin < end) {
        int pivot = array[end];

        int last_smaller_than_pivot = begin - 1;
        for (int j = begin; j < end; j++) {
            ++comparison;

            if (array[j] <= pivot) {
                ++last_smaller_than_pivot;
                ++permutations;
            }
        }
        std::swap(array[last_smaller_than_pivot + 1], array[end]);
    }
}

```

```

        std::swap(array[last_smaller_than_pivot], array[j]);
    }
}

++permutations;

std::swap(array[last_smaller_than_pivot + 1], array[end]);

int pivot_index = last_smaller_than_pivot + 1;

Sort(array, begin, pivot_index - 1);
Sort(array, pivot_index + 1, end);
}
}

void sort_matrix(std::vector<std::vector<int>>& matrix, ISort& sorting_class)
{
    for (auto& row : matrix) {
        std::vector<int> even_elements;
        for (auto& element : row) {
            if (element % 2 == 0) {
                even_elements.push_back(element);
            }
        }
        // собрать все четные

        sorting_class.Sort(even_elements); // отсортировать

        auto even_elements_iterator = even_elements.begin();

        for (auto& element : row) {
            if (element % 2 == 0) {
                element = *even_elements_iterator++;
            }
        }
        // расставить на места
    }
}

```

}
}